



UnB - FCTE

Estrutura de Dados e Algoritmos

Listas Encadeadas



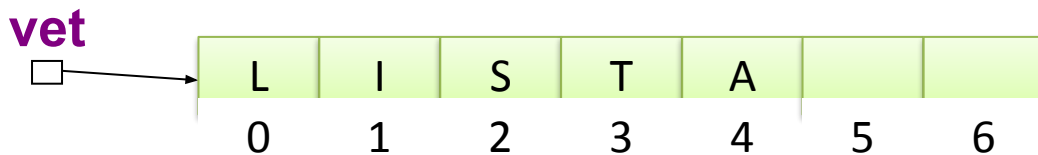
Listas

- Uma lista é uma representação de uma sequência de objetos na memória do computador;
- Cada elemento da lista é armazenado em um nó ou célula: o primeiro elemento na primeira célula, o segundo na segunda e assim por diante.



Listas Sequenciais

- Conjunto de itens organizados (vetor)
 - A organização é implícita (pela posição):

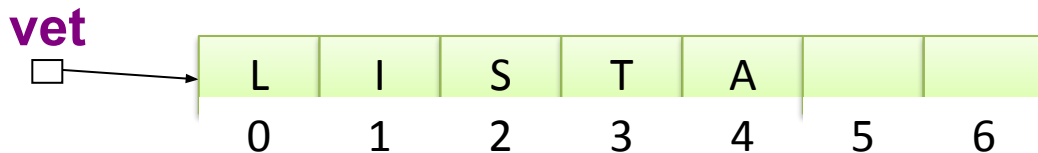


- **vet** representa a lista -> A lista está armazenada em células contíguas da memória.



Listas Sequenciais

- Desvantagens: Quantidade de Nós pré-definida:
 - memória alocada sem uso;
 - impossibilidade de alocar mais memória.





Listas Sequenciais

- Solução:
 - Utilizar Estruturas de Dados que cresçam e diminuam na medida da necessidade -> Estruturas Dinâmicas;
 - Alocação dinâmica de memória conforme demanda.

Listas Encadeadas



Estrutura de Uma Lista Encadeada

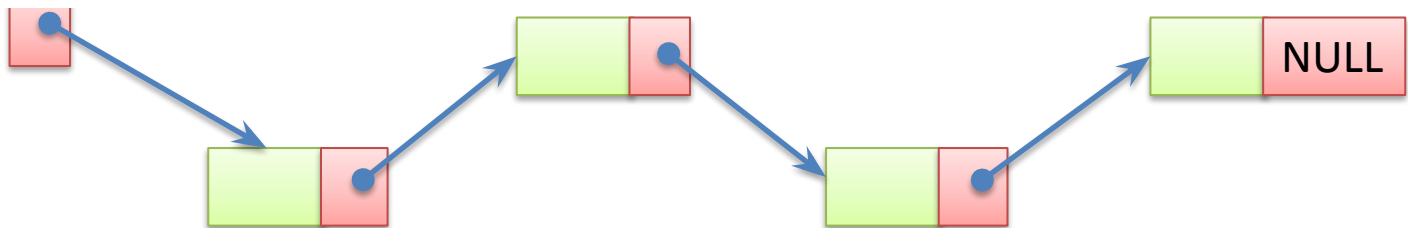
- Podem crescer e diminuir dinamicamente;
 - Tamanho máximo não precisa ser definido previamente;
 - Proveem flexibilidade, permitindo que os itens sejam rearranjados eficientemente;
 - Também chamadas de Listas Ligadas.
- O acesso a um nó é feito de forma sequencial $\rightarrow O(n)$.
Diferente de vetor $\rightarrow O(1)$;



Estrutura de Uma Lista Encadeada

- Uma lista encadeada é uma sequência de nós não contíguos;
- Cada nó contém um objeto de algum tipo e, no mínimo, o endereço do nó seguinte;
- Uma lista encadeada deve ter um ponteiro para o início da lista (primeiro nó);

Lista





Lista Encadeada – Inclusão na Cabeça

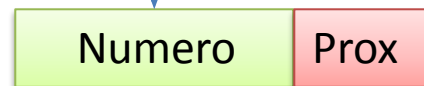
```
struct TNo
```

```
{
```

```
    int Numero;
```

```
    TNo *Prox;
```

```
};
```





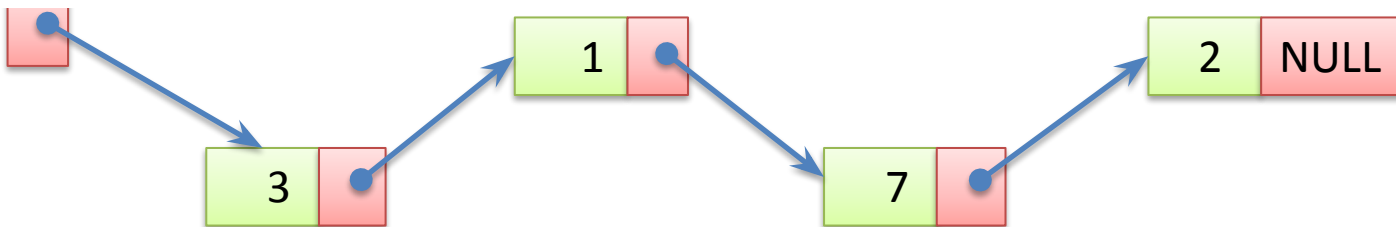
Lista Encadeada – Inclusão na Cabeça

```
1 bool Vazia(TNo *pLista)
2 {
3     if(pLista == NULL)
4         return true;
5     else
6         return false;
7 }
```

Lista

NULL

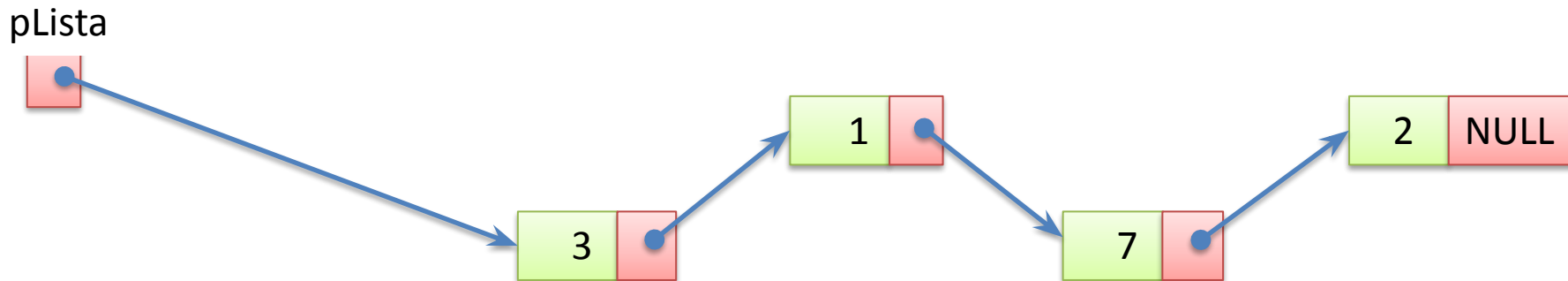
pLista





Lista Encadeada – Inclusão na Cabeça

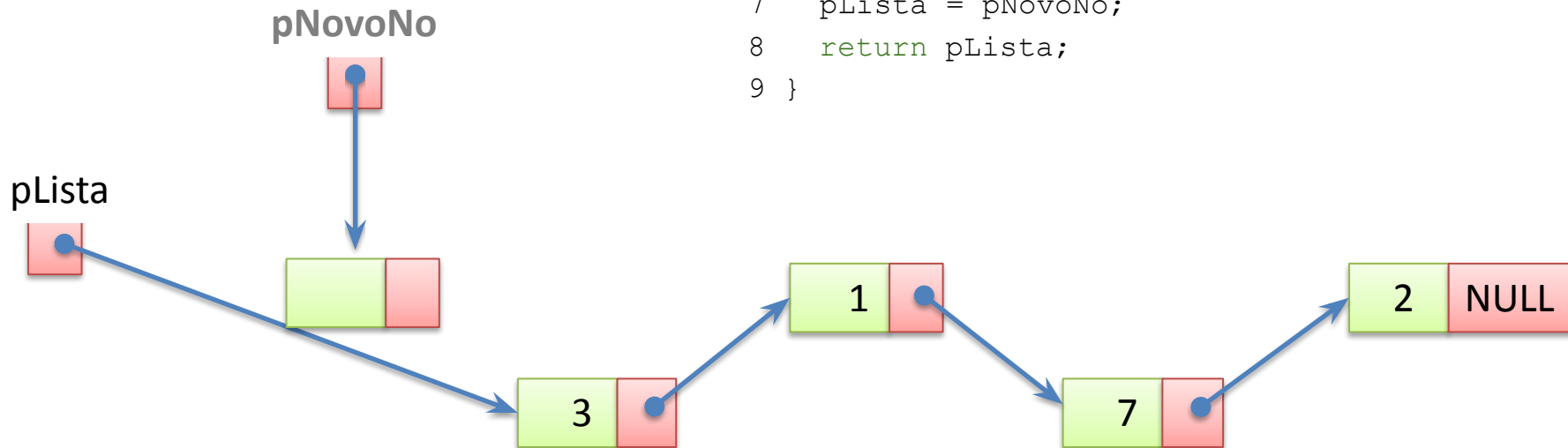
```
1 TNo *IncluiCabeca(TNo *pLista, int pValor)
2 {
3     TNo *pNovoNo;
4     pNovoNo = (TNo *) malloc(sizeof(TNo));
5     pNovoNo->Numero = pValor;
6     pNovoNo->Prox = pLista;
7     pLista = pNovoNo;
8     return pLista;
9 }
```





Lista Encadeada – Inclusão na Cabeça

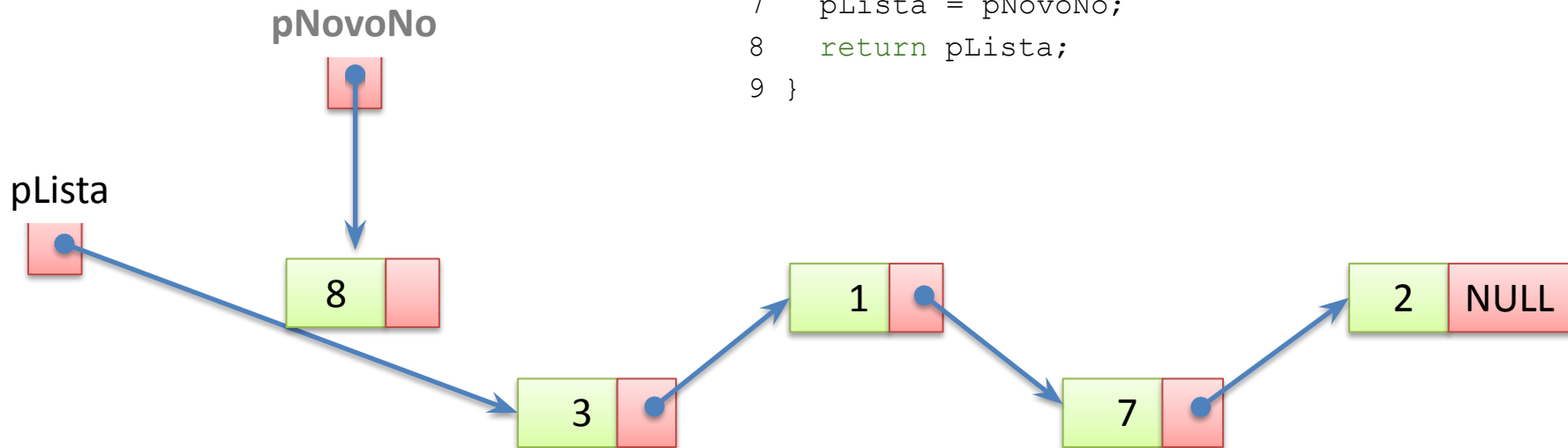
```
1 TNo *IncluiCabeca(TNo *pLista, int pValor)
2 {
3     TNo *pNovoNo;
4     pNovoNo = (TNo *) malloc(sizeof(TNo));
5     pNovoNo->Numero = pValor;
6     pNovoNo->Prox = pLista;
7     pLista = pNovoNo;
8     return pLista;
9 }
```





Lista Encadeada – Inclusão na Cabeça

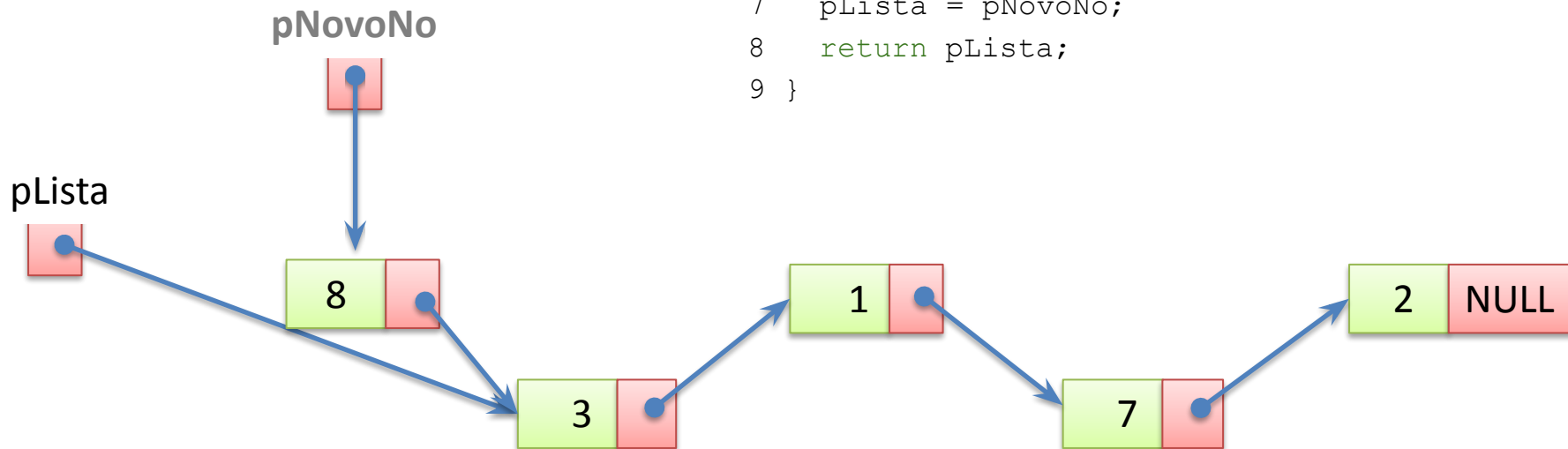
```
1 TNo *IncluiCabeça(TNo *pLista, int pValor)
2 {
3     TNo *pNovoNo;
4     pNovoNo = (TNo *) malloc(sizeof(TNo));
5     pNovoNo->Numero = pValor;
6     pNovoNo->Prox = pLista;
7     pLista = pNovoNo;
8     return pLista;
9 }
```





Lista Encadeada – Inclusão na Cabeça

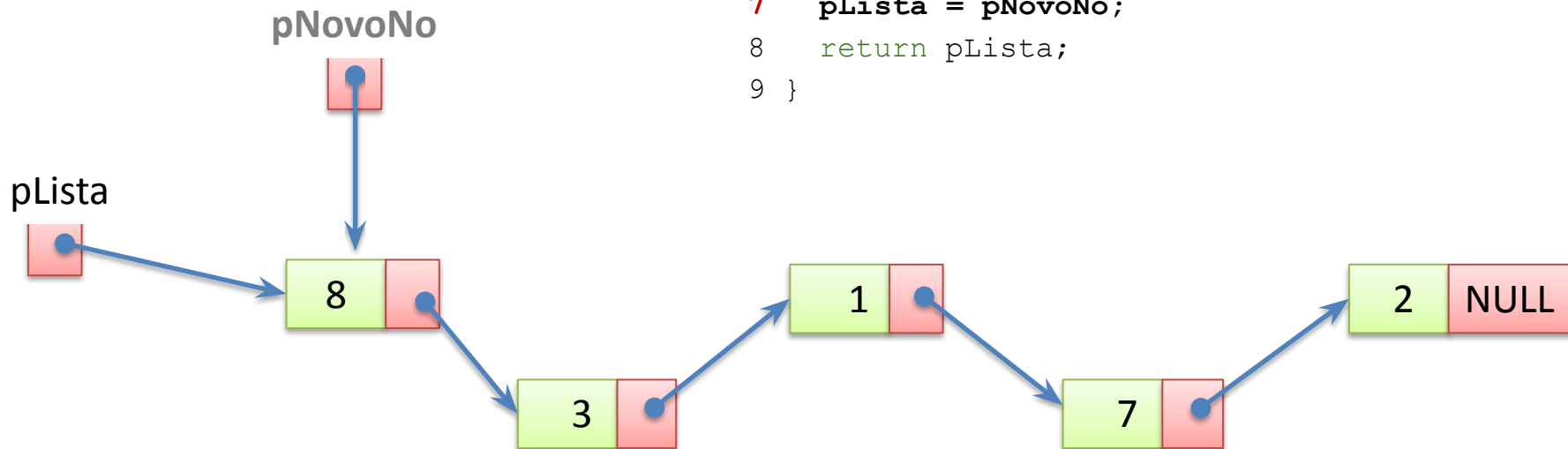
```
1 TNo *IncluiCabeça(TNo *pLista, int pValor)
2 {
3     TNo *pNovoNo;
4     pNovoNo = (TNo *) malloc(sizeof(TNo));
5     pNovoNo->Numero = pValor;
6     pNovoNo->Prox = pLista;
7     pLista = pNovoNo;
8     return pLista;
9 }
```





Lista Encadeada – Inclusão na Cabeça

```
1 TNo *IncluiCabeça(TNo *pLista, int pValor)
2 {
3     TNo *pNovoNo;
4     pNovoNo = (TNo *) malloc(sizeof(TNo));
5     pNovoNo->Numero = pValor;
6     pNovoNo->Prox = pLista;
7     pLista = pNovoNo;
8     return pLista;
9 }
```

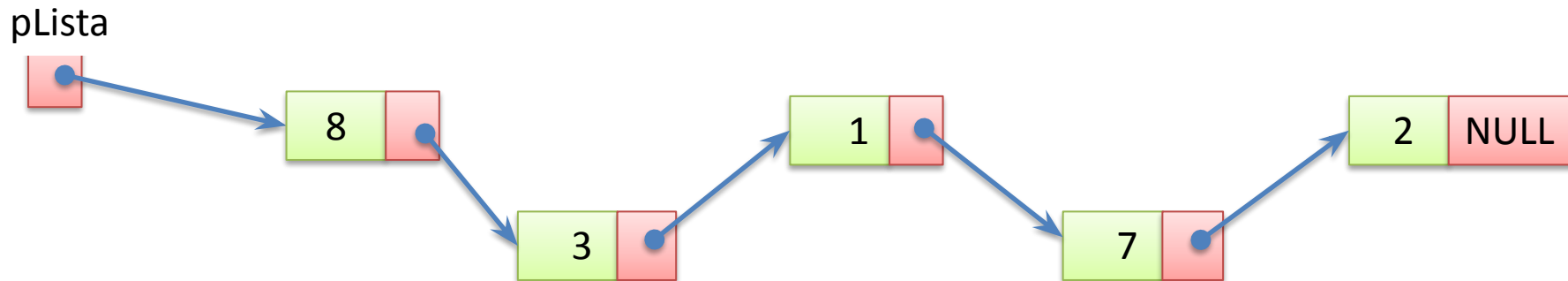




Lista Encadeada – Inclusão na Cabeça

$O(1)$

```
1 TNo *IncluiCabeça(TNo *pLista, int pValor)
2 {
3     TNo *pNovoNo;
4     pNovoNo = (TNo *) malloc(sizeof(TNo));
5     pNovoNo->Numero = pValor;
6     pNovoNo->Prox = pLista;
7     pLista = pNovoNo;
8     return pLista;
9 }
```

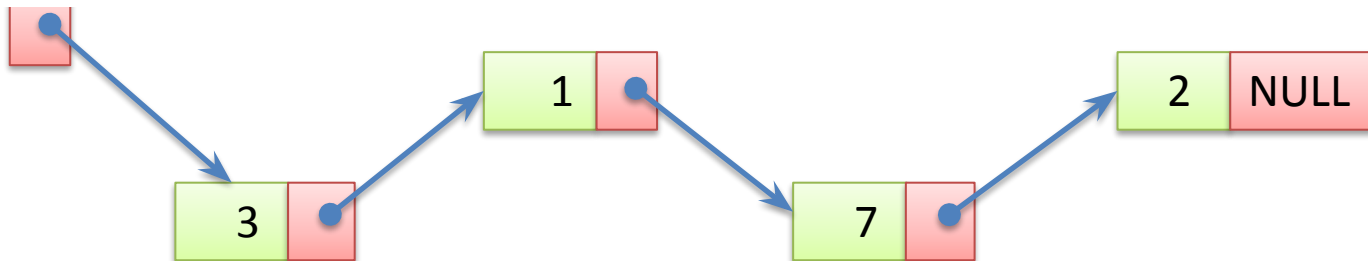




Lista Encadeada – Inclusão na Calda

```
1 TNo *IncluiCalda(TNo *pLista, int pValor)
2 {
3     TNo *pNovoNo, *pAux;
4     pNovoNo = (TNo *) malloc(sizeof(TNo));
5     pNovoNo->Numero = pValor;
6     pNovoNo->Prox = NULL;
7     pAux = pLista;
8     while (pAux->Prox != NULL)
9         pAux = pAux->Prox;
10    pAux->Prox = pNovoNo;
11    return pLista;
12 }
```

pLista

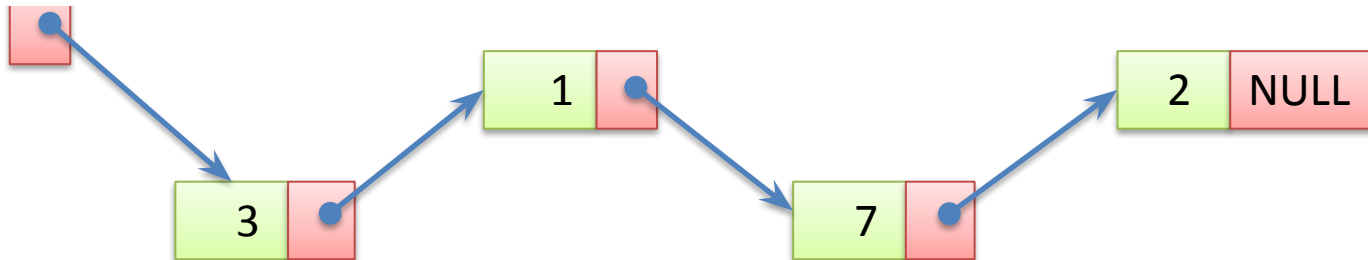




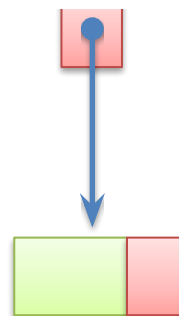
Lista Encadeada – Inclusão na Calda

```
1 TNo *IncluiCalda(TNo *pLista, int pValor)
2 {
3     TNo *pNovoNo, *pAux;
4     pNovoNo = (TNo *) malloc(sizeof(TNo));
5     pNovoNo->Numero = pValor;
6     pNovoNo->Prox = NULL;
7     pAux = pLista;
8     while (pAux->Prox != NULL)
9         pAux = pAux->Prox;
10    pAux->Prox = pNovoNo;
11    return pLista;
12 }
```

pLista



pNovoNo

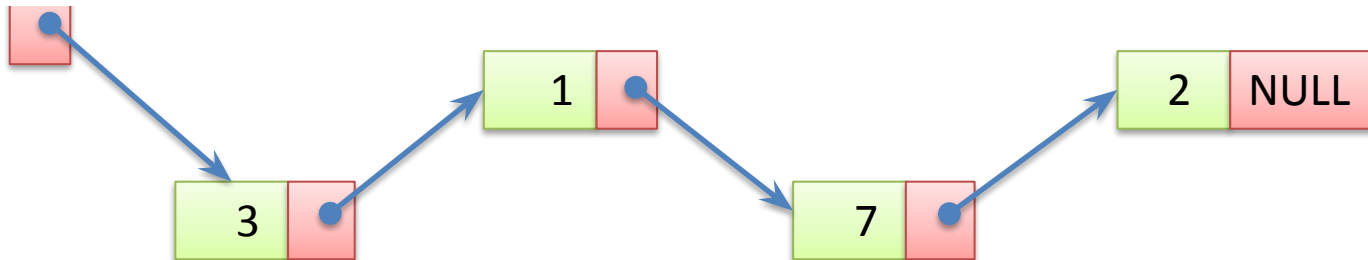




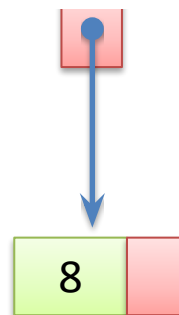
Lista Encadeada – Inclusão na Calda

```
1 TNo *IncluiCalda(TNo *pLista, int pValor)
2 {
3     TNo *pNovoNo, *pAux;
4     pNovoNo = (TNo *) malloc(sizeof(TNo));
5     pNovoNo->Numero = pValor;
6     pNovoNo->Prox = NULL;
7     pAux = pLista;
8     while (pAux->Prox != NULL)
9         pAux = pAux->Prox;
10    pAux->Prox = pNovoNo;
11    return pLista;
12 }
```

pLista



pNovoNo

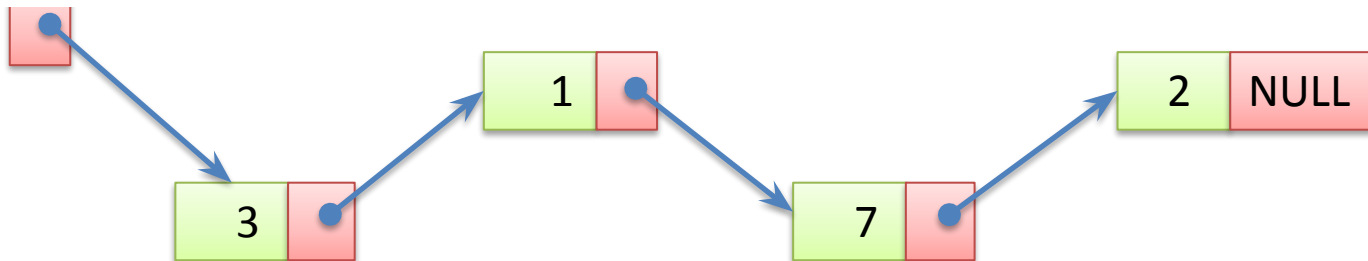




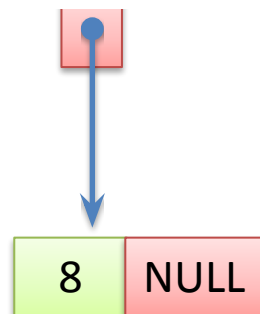
Lista Encadeada – Inclusão na Calda

```
1 TNo *IncluiCalda(TNo *pLista, int pValor)
2 {
3     TNo *pNovoNo, *pAux;
4     pNovoNo = (TNo *) malloc(sizeof(TNo));
5     pNovoNo->Numero = pValor;
6     pNovoNo->Prox = NULL;
7     pAux = pLista;
8     while (pAux->Prox != NULL)
9         pAux = pAux->Prox;
10    pAux->Prox = pNovoNo;
11    return pLista;
12 }
```

pLista



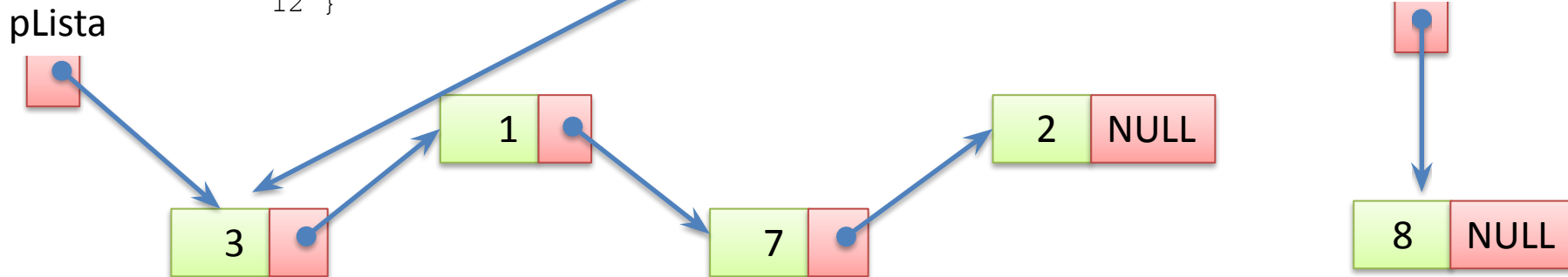
pNovoNo





Lista Encadeada – Inclusão na Calda

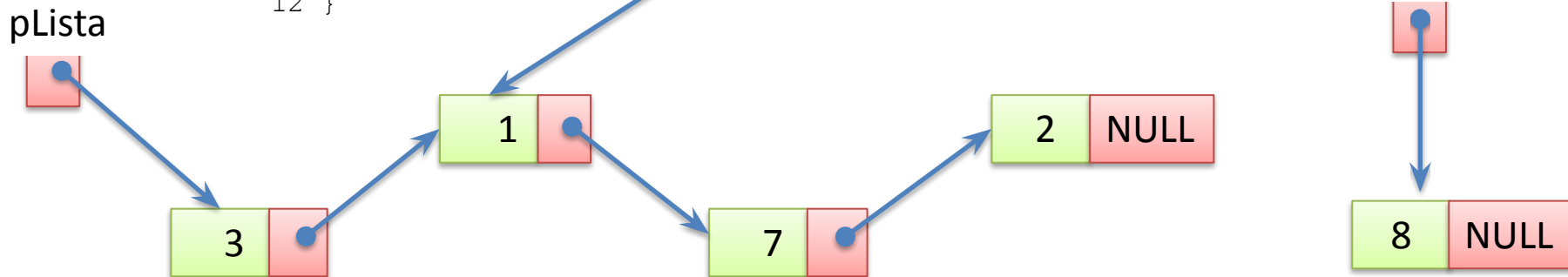
```
1 TNo *IncluiCalda(TNo *pLista, int pValor)
2 {
3     TNo *pNovoNo, *pAux;
4     pNovoNo = (TNo *) malloc(sizeof(TNo));
5     pNovoNo->Numero = pValor;
6     pNovoNo->Prox = NULL;
7     pAux = pLista;
8     while (pAux->Prox != NULL)
9         pAux = pAux->Prox;
10    pAux->Prox = pNovoNo;
11    return pLista;
12 }
```





Lista Encadeada – Inclusão na Calda

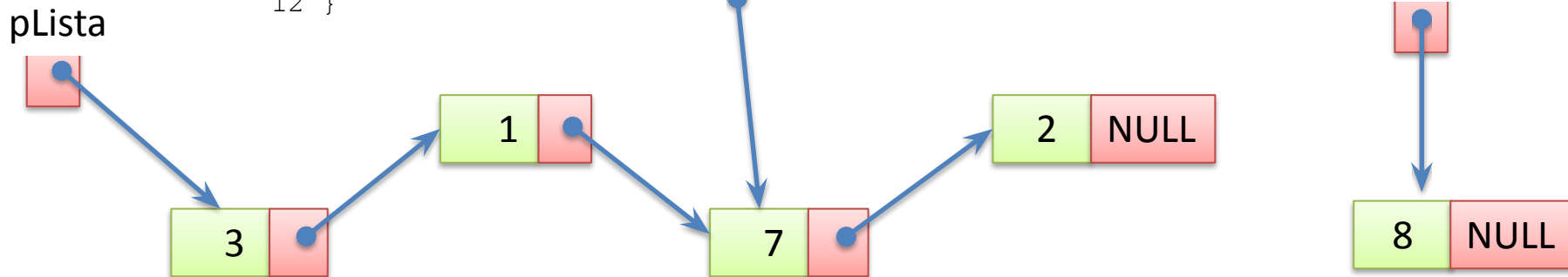
```
1 TNo *IncluiCalda(TNo *pLista, int pValor)
2 {
3     TNo *pNovoNo, *pAux;
4     pNovoNo = (TNo *) malloc(sizeof(TNo));
5     pNovoNo->Numero = pValor;
6     pNovoNo->Prox = NULL;
7     pAux = pLista;
8     while (pAux->Prox != NULL)
9         pAux = pAux->Prox;
10    pAux->Prox = pNovoNo;
11    return pLista;
12 }
```





Lista Encadeada – Inclusão na Calda

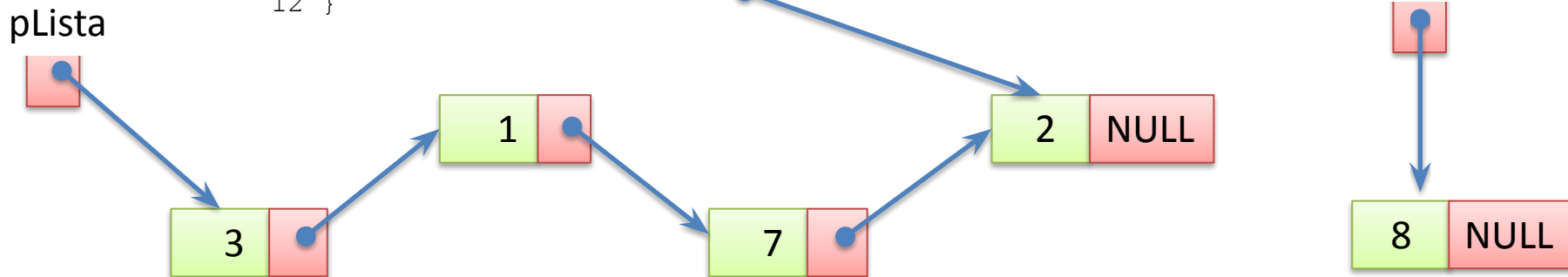
```
1 TNo *IncluiCalda(TNo *pLista, int pValor)
2 {
3     TNo *pNovoNo, *pAux;
4     pNovoNo = (TNo *) malloc(sizeof(TNo));
5     pNovoNo->Numero = pValor;
6     pNovoNo->Prox = NULL;
7     pAux = pLista;
8     while (pAux->Prox != NULL)
9         pAux = pAux->Prox;
10    pAux->Prox = pNovoNo;
11    return pLista;
12 }
```





Lista Encadeada – Inclusão na Calda

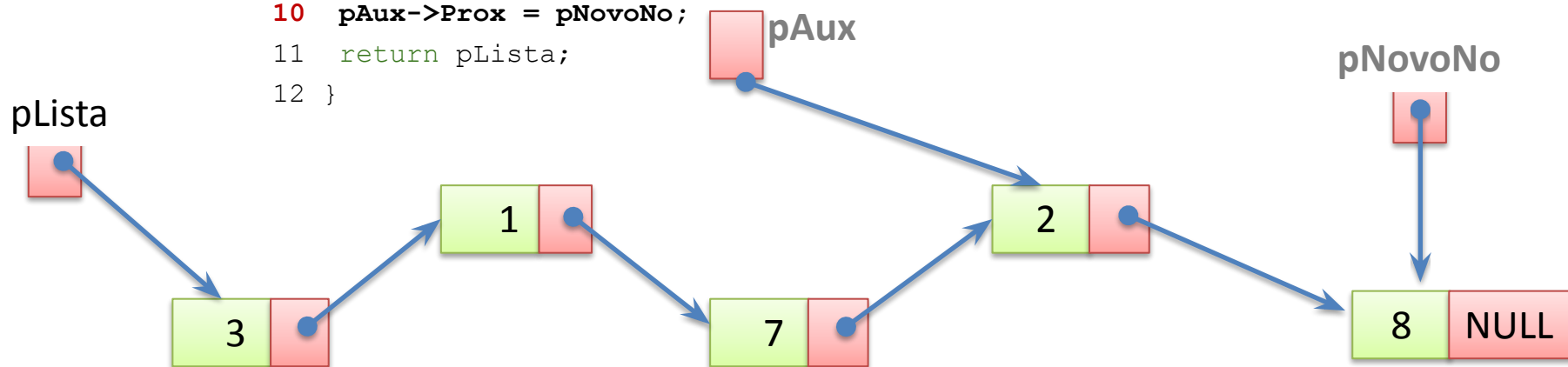
```
1 TNo *IncluiCalda(TNo *pLista, int pValor)
2 {
3     TNo *pNovoNo, *pAux;
4     pNovoNo = (TNo *) malloc(sizeof(TNo));
5     pNovoNo->Numero = pValor;
6     pNovoNo->Prox = NULL;
7     pAux = pLista;
8     while (pAux->Prox != NULL)
9         pAux = pAux->Prox;
10    pAux->Prox = pNovoNo;
11    return pLista;
12 }
```





Lista Encadeada – Inclusão na Calda

```
1 TNo *IncluiCalda(TNo *pLista, int pValor)
2 {
3     TNo *pNovoNo, *pAux;
4     pNovoNo = (TNo *) malloc(sizeof(TNo));
5     pNovoNo->Numero = pValor;
6     pNovoNo->Prox = NULL;
7     pAux = pLista;
8     while (pAux->Prox != NULL)
9         pAux = pAux->Prox;
10    pAux->Prox = pNovoNo;
11    return pLista;
12 }
```

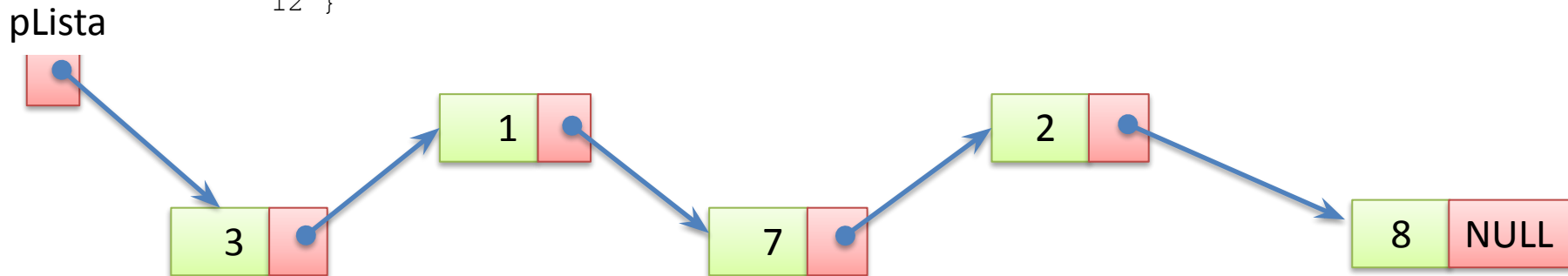




Lista Encadeada – Inclusão na Calda

```
1 TNo *IncluiCalda(TNo *pLista, int pValor)
2 {
3     TNo *pNovoNo, *pAux;
4     pNovoNo = (TNo *) malloc(sizeof(TNo));
5     pNovoNo->Numero = pValor;
6     pNovoNo->Prox = NULL;
7     pAux = pLista;
8     while (pAux->Prox != NULL)
9         pAux = pAux->Prox;
10    pAux->Prox = pNovoNo;
11    return pLista;
12 }
```

$O(n)$



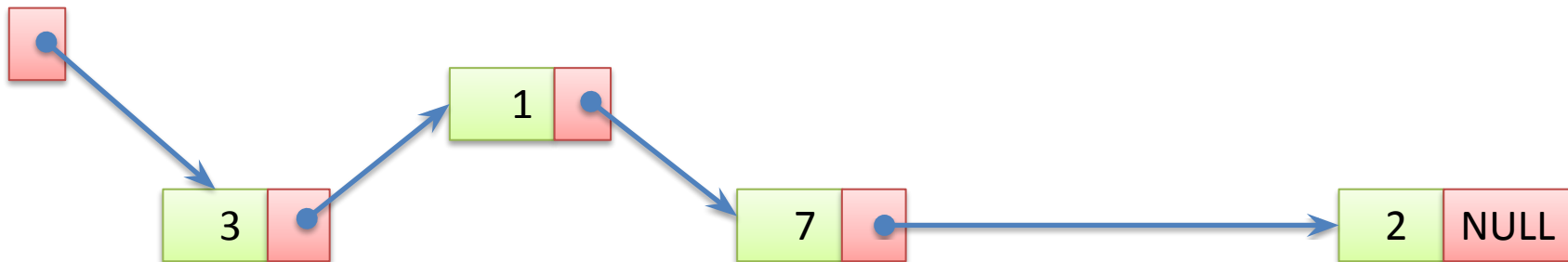


Lista Encadeada – Inclusão Antes de Uma Chave

pChave = 2

```
1 TNo *IncluiAntes(TNo *pLista, int pChave, int pValor)
2 {
3     TNo *pNovoNo, *pAux;
4     pNovoNo = (TNo *) malloc(sizeof(TNo));
5     pNovoNo->Numero = pValor;
6     pAux = pLista;
7     while (pAux->Prox->Valor != pChave)
8         pAux = pAux->Prox;
9     pNovoNo->Prox = pAux->Prox;
10    pAux->Prox = pNovoNo;
11    return pLista;
12 }
```

pLista



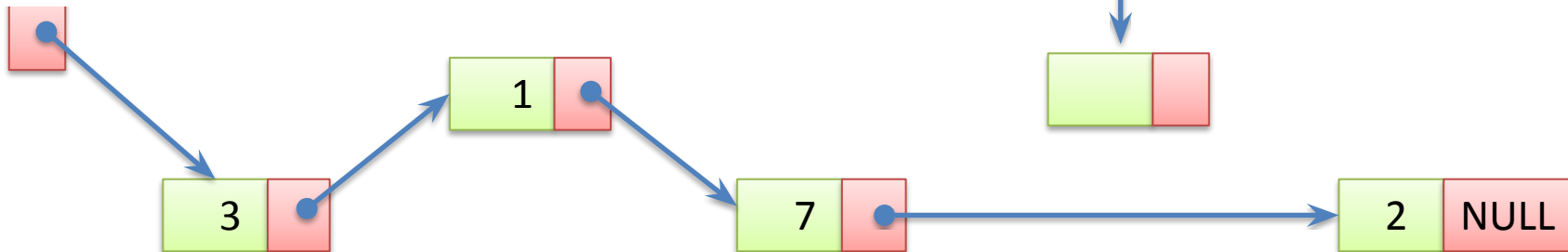


Lista Encadeada – Inclusão Antes de Uma Chave

pChave = 2

```
1 TNo *IncluiAntes(TNo *pLista, int pChave, int pValor)
2 {
3     TNo *pNovoNo, *pAux;
4     pNovoNo = (TNo *) malloc(sizeof(TNo));
5     pNovoNo->Numero = pValor;
6     pAux = pLista;
7     while (pAux->Prox->Valor != pChave)
8         pAux = pAux->Prox;
9     pNovoNo->Prox = pAux->Prox;
10    pAux->Prox = pNovoNo;
11    return pLista;
12 }
```

pLista



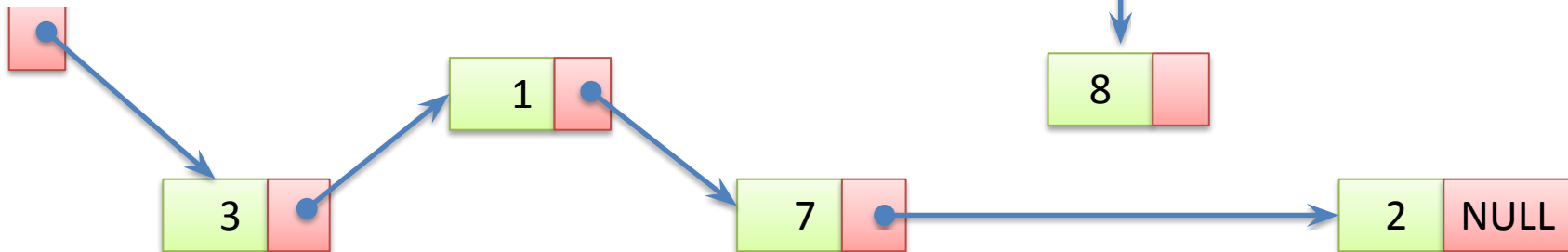


Lista Encadeada – Inclusão Antes de Uma Chave

pChave = 2

```
1 TNo *IncluiAntes(TNo *pLista, int pChave, int pValor)
2 {
3     TNo *pNovoNo, *pAux;
4     pNovoNo = (TNo *) malloc(sizeof(TNo));
5     pNovoNo->Numero = pValor;
6     pAux = pLista;
7     while (pAux->Prox->Valor != pChave)
8         pAux = pAux->Prox;
9     pNovoNo->Prox = pAux->Prox;
10    pAux->Prox = pNovoNo;
11    return pLista;
12 }
```

pLista



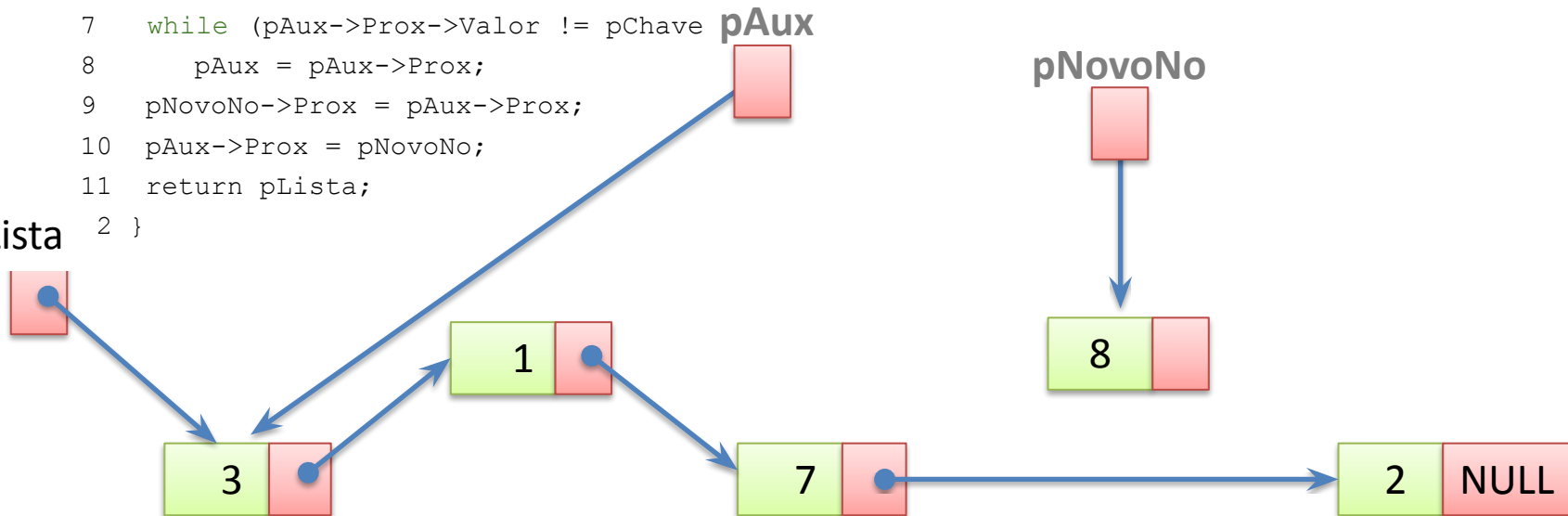


Lista Encadeada – Inclusão Antes de Uma Chave

pChave = 2

```
1 TNo *IncluiAntes(TNo *pLista, int pChave, int pValor)
2 {
3     TNo *pNovoNo, *pAux;
4     pNovoNo = (TNo *) malloc(sizeof(TNo));
5     pNovoNo->Numero = pValor;
6     pAux = pLista;
7     while (pAux->Prox->Valor != pChave pAux
8         pAux = pAux->Prox;
9     pNovoNo->Prox = pAux->Prox;
10    pAux->Prox = pNovoNo;
11    return pLista;
12 }
```

pLista



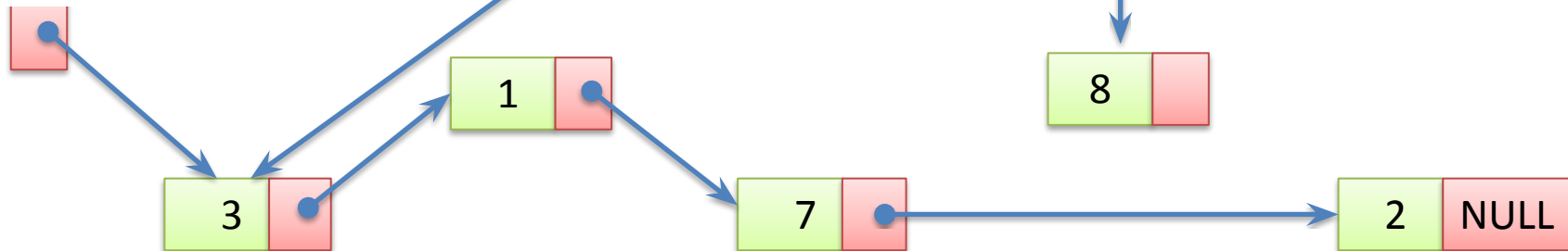


Lista Encadeada – Inclusão Antes de Uma Chave

pChave = 2

```
1 TNo *IncluiAntes(TNo *pLista, int pChave, int pValor)
2 {
3     TNo *pNovoNo, *pAux;
4     pNovoNo = (TNo *) malloc(sizeof(TNo));
5     pNovoNo->Numero = pValor;
6     pAux = pLista;
7     while (pAux->Prox->Valor != pChave pAux
8         pAux = pAux->Prox;
9     pNovoNo->Prox = pAux->Prox;
10    pAux->Prox = pNovoNo;
11    return pLista;
12 }
```

pLista



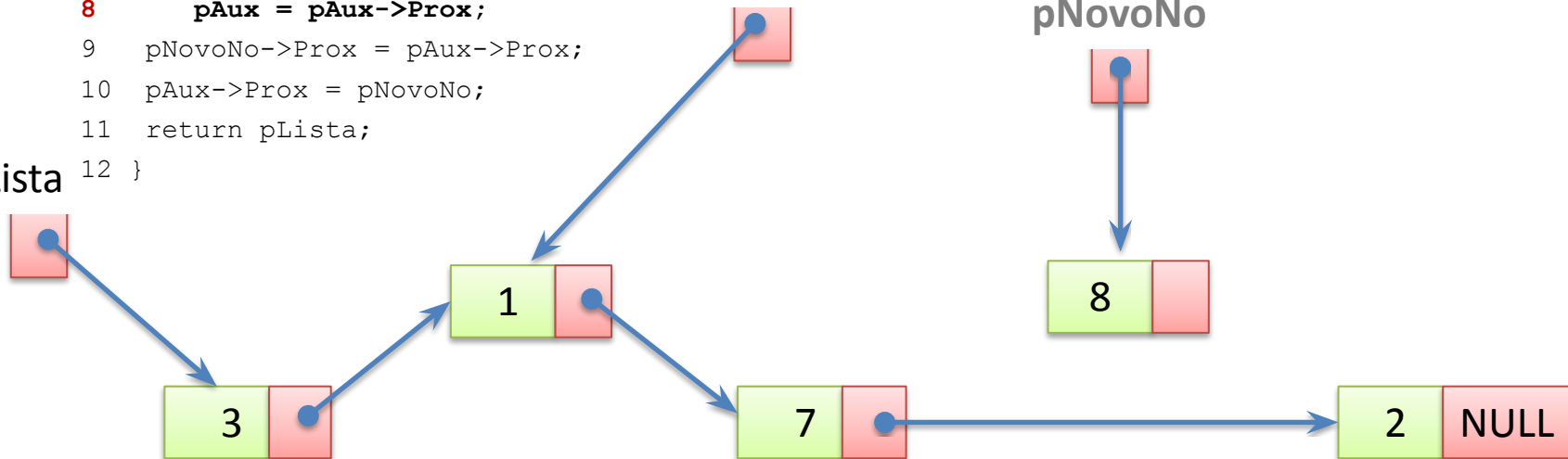


Lista Encadeada – Inclusão Antes de Uma Chave

pChave = 2

```
1 TNo *IncluiAntes(TNo *pLista, int pChave, int pValor)
2 {
3     TNo *pNovoNo, *pAux;
4     pNovoNo = (TNo *) malloc(sizeof(TNo));
5     pNovoNo->Numero = pValor;
6     pAux = pLista;
7     while (pAux->Prox->Valor != pChave) pAux
8         pAux = pAux->Prox;
9     pNovoNo->Prox = pAux->Prox;
10    pAux->Prox = pNovoNo;
11    return pLista;
12 }
```

pLista



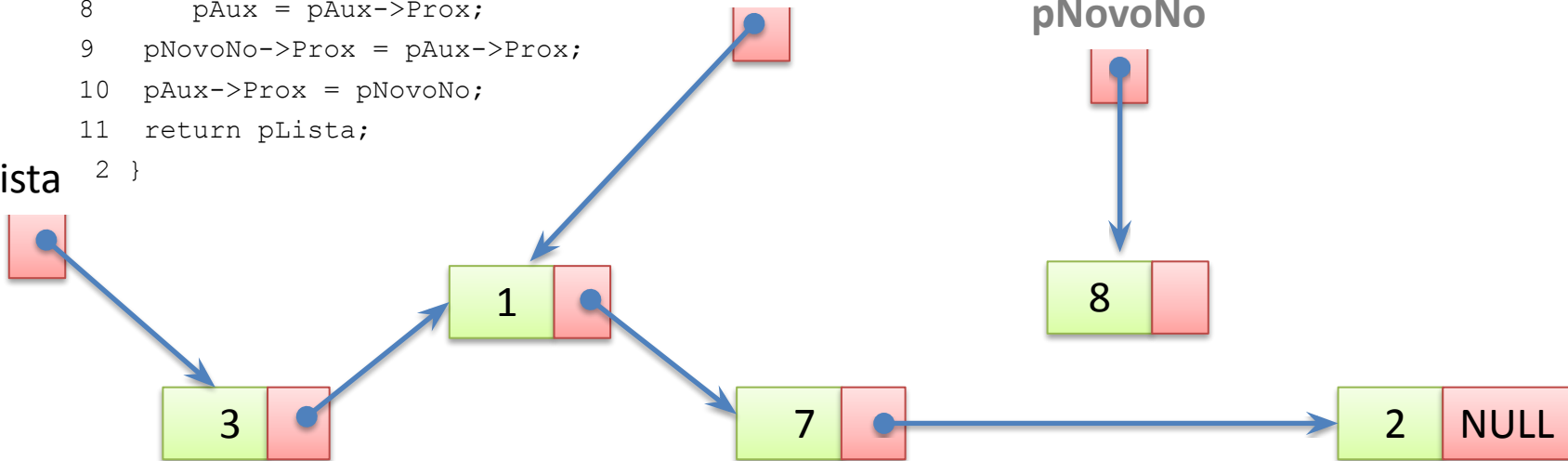


Lista Encadeada – Inclusão Antes de Uma Chave

pChave = 2

```
1 TNo *IncluiAntes(TNo *pLista, int pChave, int pValor)
2 {
3     TNo *pNovoNo, *pAux;
4     pNovoNo = (TNo *) malloc(sizeof(TNo));
5     pNovoNo->Numero = pValor;
6     pAux = pLista;
7     while (pAux->Prox->Valor != pChave) pAux
8         pAux = pAux->Prox;
9     pNovoNo->Prox = pAux->Prox;
10    pAux->Prox = pNovoNo;
11    return pLista;
12 }
```

pLista



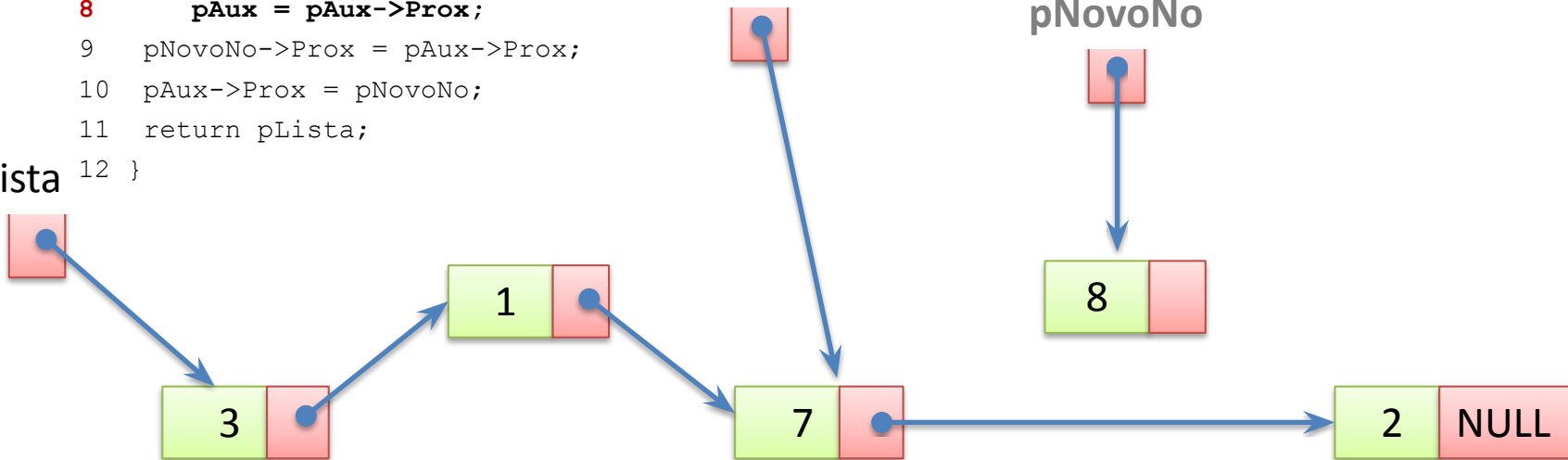


Lista Encadeada – Inclusão Antes de Uma Chave

pChave = 2

```
1 TNo *IncluiAntes(TNo *pLista, int pChave, int pValor)
2 {
3     TNo *pNovoNo, *pAux;
4     pNovoNo = (TNo *) malloc(sizeof(TNo));
5     pNovoNo->Numero = pValor;
6     pAux = pLista;
7     while (pAux->Prox->Valor != pChave) pAux
8         pAux = pAux->Prox;
9     pNovoNo->Prox = pAux->Prox;
10    pAux->Prox = pNovoNo;
11    return pLista;
12 }
```

pLista



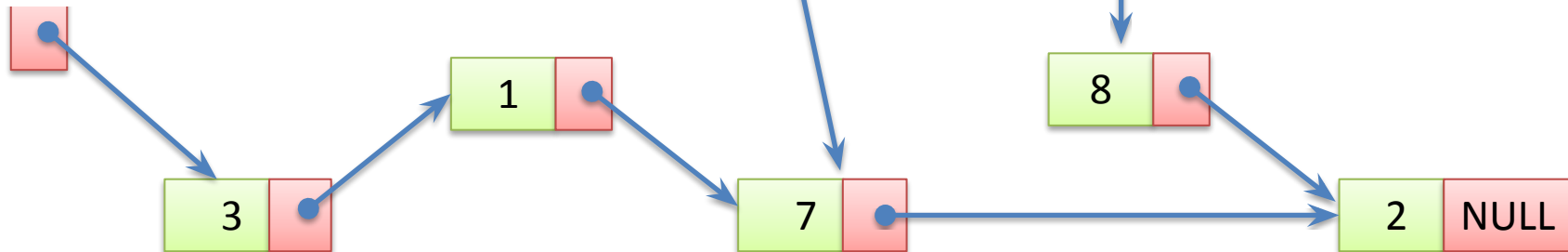


Lista Encadeada – Inclusão Antes de Uma Chave

pChave = 2

```
1 TNo *IncluiAntes(TNo *pLista, int pChave, int pValor)
2 {
3     TNo *pNovoNo, *pAux;
4     pNovoNo = (TNo *) malloc(sizeof(TNo));
5     pNovoNo->Numero = pValor;
6     pAux = pLista;
7     while (pAux->Prox->Valor != pChave) pAux
8         pAux = pAux->Prox;
9     pNovoNo->Prox = pAux->Prox;
10    pAux->Prox = pNovoNo;
11    return pLista;
12 }
```

pLista



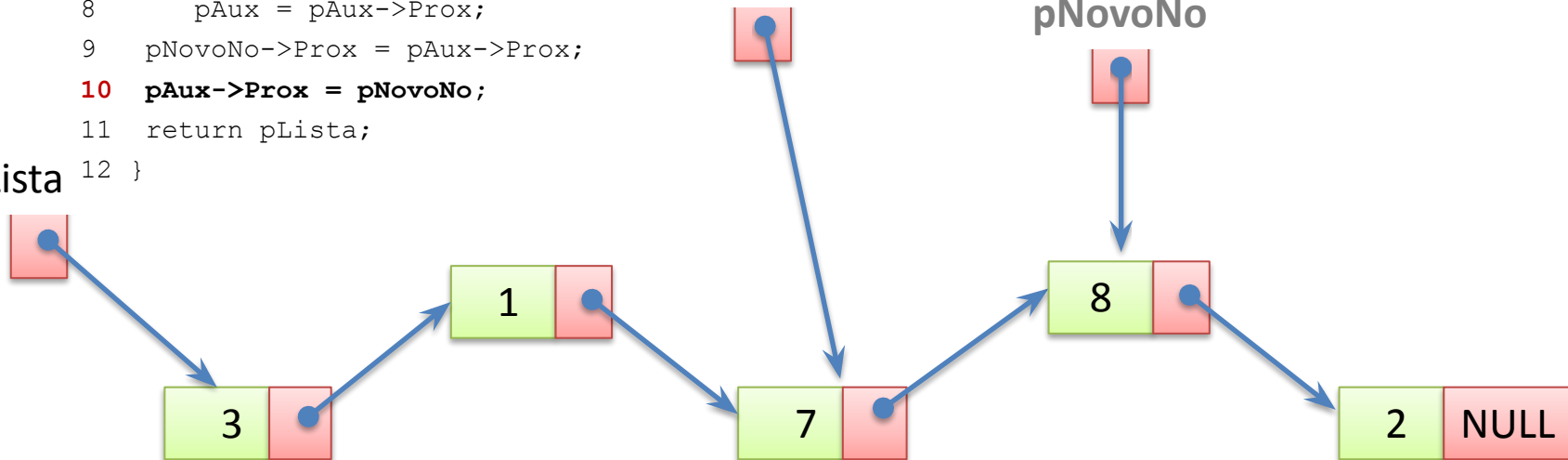


Lista Encadeada – Inclusão Antes de Uma Chave

pChave = 2

```
1 TNo *IncluiAntes(TNo *pLista, int pChave, int pValor)
2 {
3     TNo *pNovoNo, *pAux;
4     pNovoNo = (TNo *) malloc(sizeof(TNo));
5     pNovoNo->Numero = pValor;
6     pAux = pLista;
7     while (pAux->Prox->Valor != pChave) pAux
8         pAux = pAux->Prox;
9     pNovoNo->Prox = pAux->Prox;
10    pAux->Prox = pNovoNo;
11    return pLista;
12 }
```

pLista





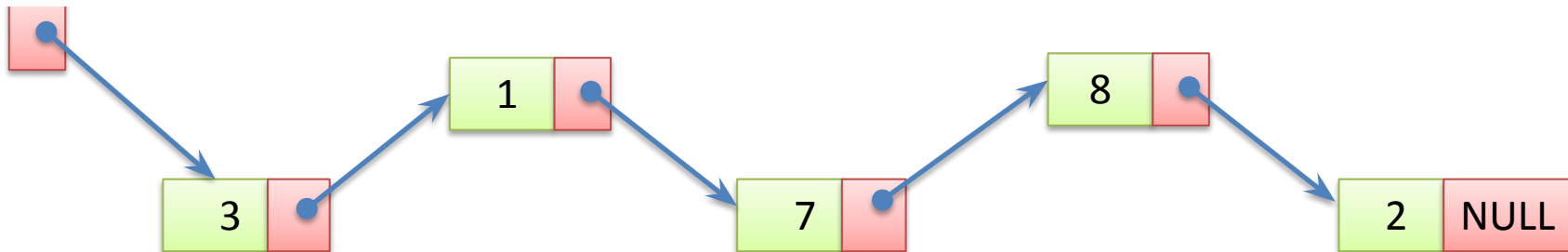
Lista Encadeada – Inclusão Antes de Uma Chave

pChave = 2

```
1 TNo *IncluiAntes(TNo *pLista, int pChave, int pValor)
2 {
3     TNo *pNovoNo, *pAux;
4     pNovoNo = (TNo *) malloc(sizeof(TNo));
5     pNovoNo->Numero = pValor;
6     pAux = pLista;
7     while (pAux->Prox->Valor != pChave)
8         pAux = pAux->Prox;
9     pNovoNo->Prox = pAux->Prox;
10    pAux->Prox = pNovoNo;
11    return pLista;
12 }
```

$O(n)$

pLista

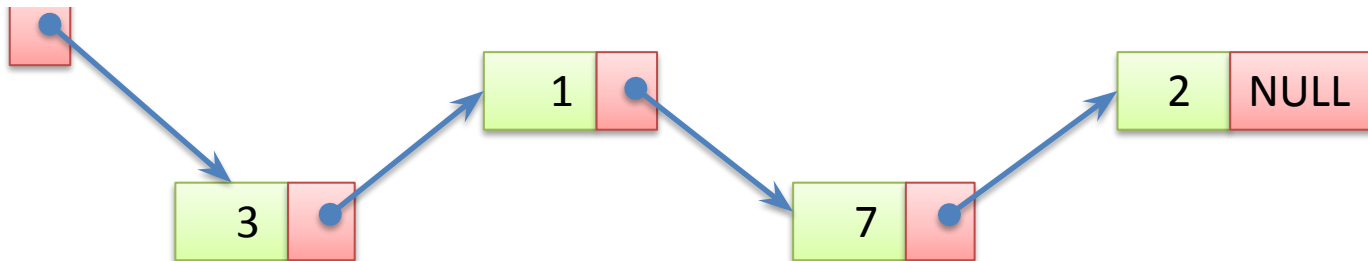




Lista Encadeada – Exclusão na Cabeça

```
1 TNo *ExcluiCabeça(TNo *pLista)
2 {
3   TNo *pAux;
4   pAux = pLista;
5   pLista = pLista->Prox;
6   free(pAux);
7   return pLista;
8 }
```

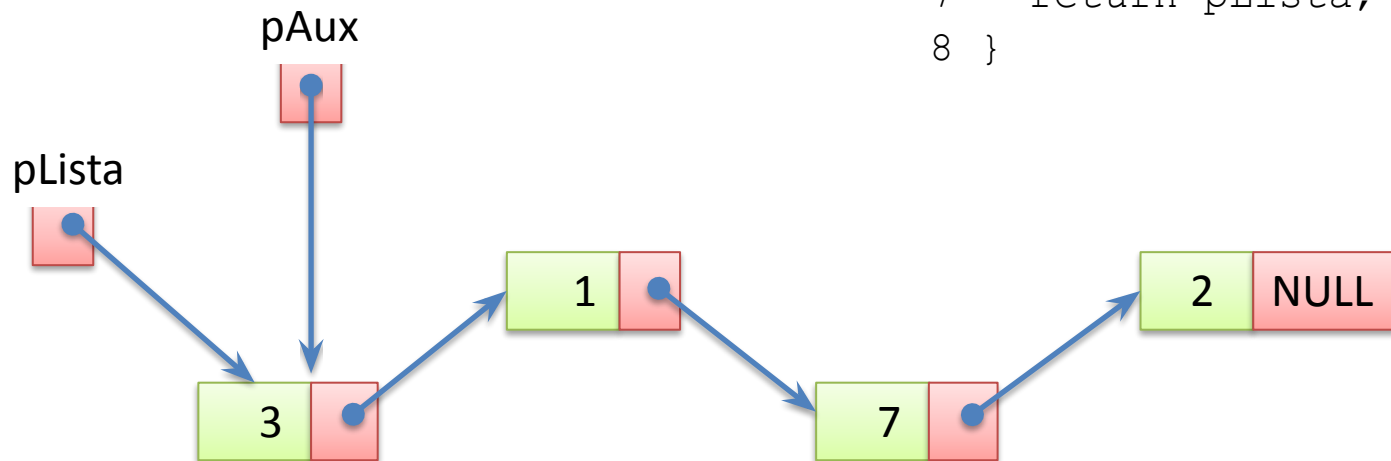
pLista





Lista Encadeada – Exclusão na Cabeça

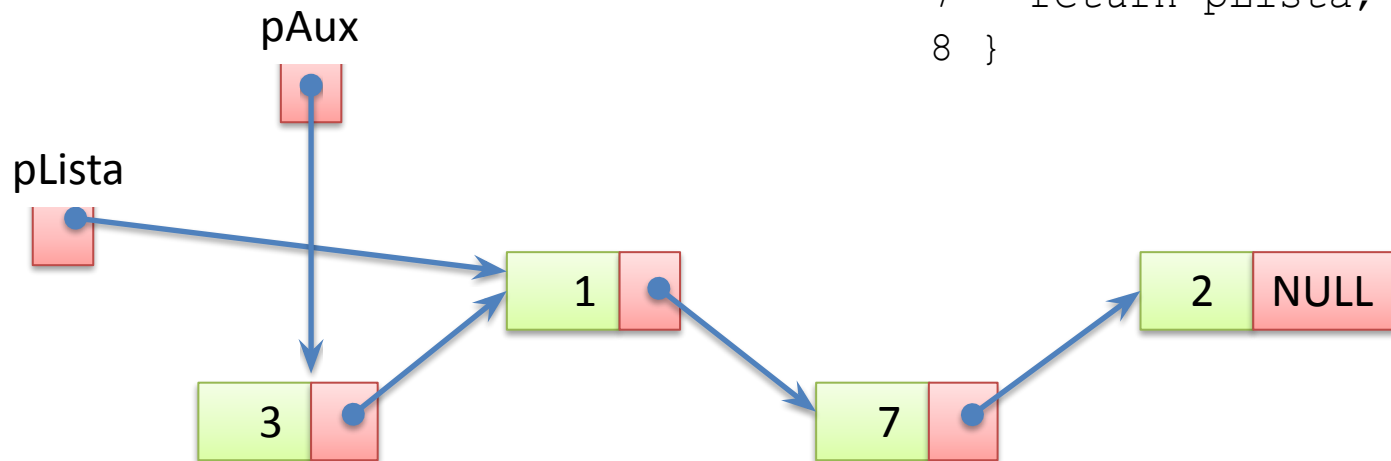
```
1 TNo *ExcluiCabeça(TNo *pLista)
2 {
3   TNo *pAux;
4   pAux = pLista;
5   pLista = pLista->Prox;
6   free(pAux);
7   return pLista;
8 }
```





Lista Encadeada – Exclusão na Cabeça

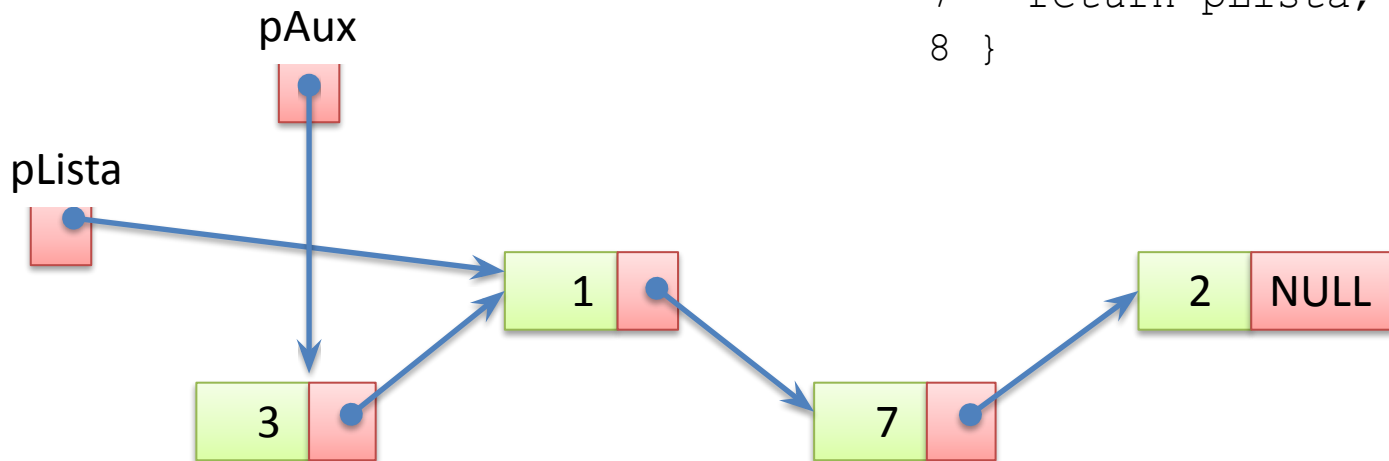
```
1 TNo *ExcluiCabeça(TNo *pLista)
2 {
3   TNo *pAux;
4   pAux = pLista;
5   pLista = pLista->Prox;
6   free(pAux);
7   return pLista;
8 }
```





Lista Encadeada – Exclusão na Cabeça

```
1 TNo *ExcluiCabeça(TNo *pLista)
2 {
3   TNo *pAux;
4   pAux = pLista;
5   pLista = pLista->Prox;
6   free (pAux) ;
7   return pLista;
8 }
```

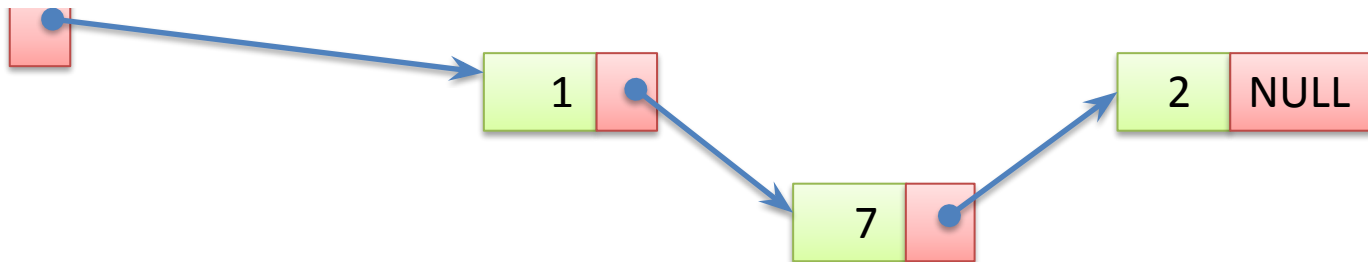




Lista Encadeada – Exclusão na Cabeça

```
1 TNo *ExcluiCabeça(TNo *pLista)
2 {
3     TNo *pAux;
4     pAux = pLista;
5     pLista = pLista->Prox;
6     free(pAux);
7     return pLista;
8 }
```

pLista

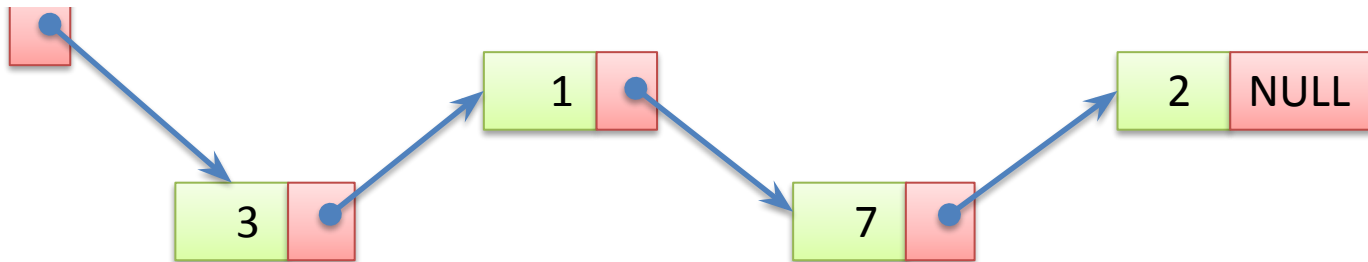




Lista Encadeada – Exclusão na Calda

```
1 TNo *ExcluiCalda(TNo *pLista)
2 {
3     TNo *pAux;
4     pAux = pLista;
5     while (pAux->Prox->Prox != NULL)
6         pAux = pAux->Prox;
7     free(pAux->Prox);
8     pAux->Prox = NULL;
9     return pLista;
10 }
```

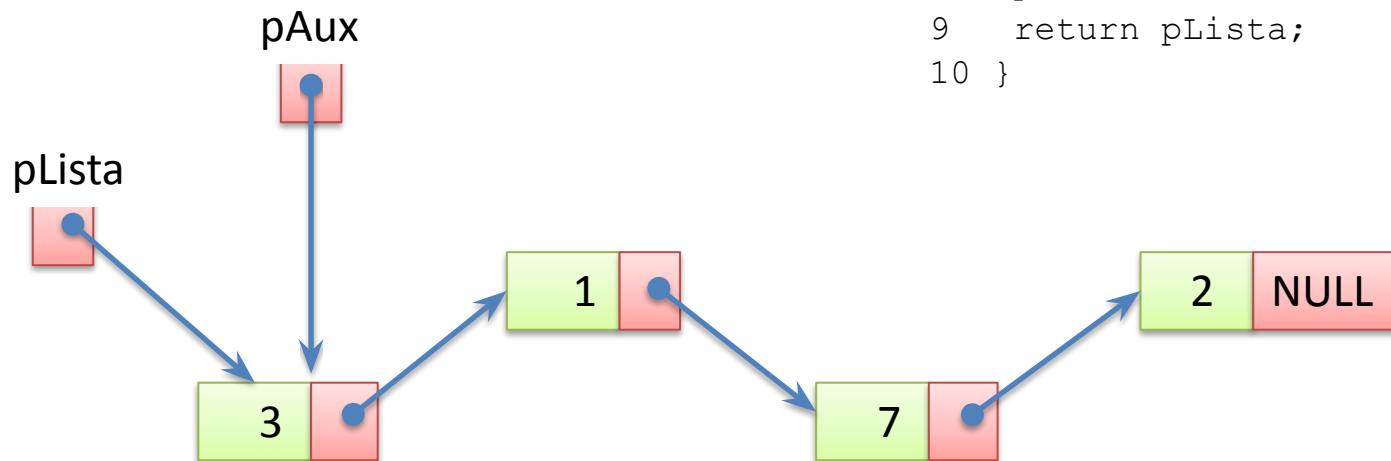
pLista





Lista Encadeada – Exclusão na Calda

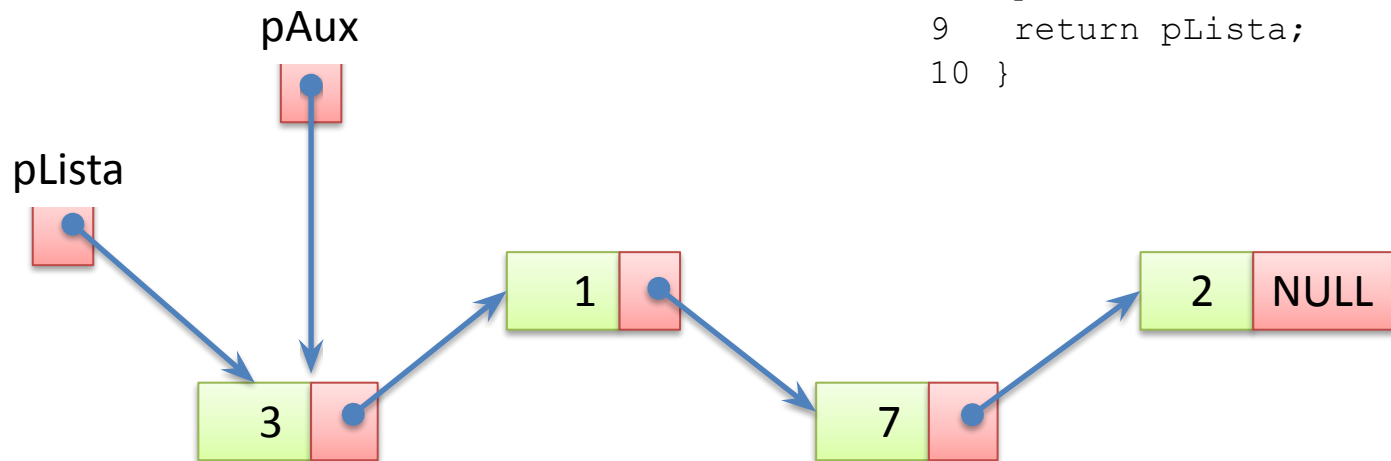
```
1 TNo *ExcluiCalda(TNo *pLista)
2 {
3     TNo *pAux;
4     pAux = pLista;
5     while (pAux->Prox->Prox != NULL)
6         pAux = pAux->Prox;
7     free(pAux->Prox);
8     pAux->Prox = NULL;
9     return pLista;
10 }
```





Lista Encadeada – Exclusão na Calda

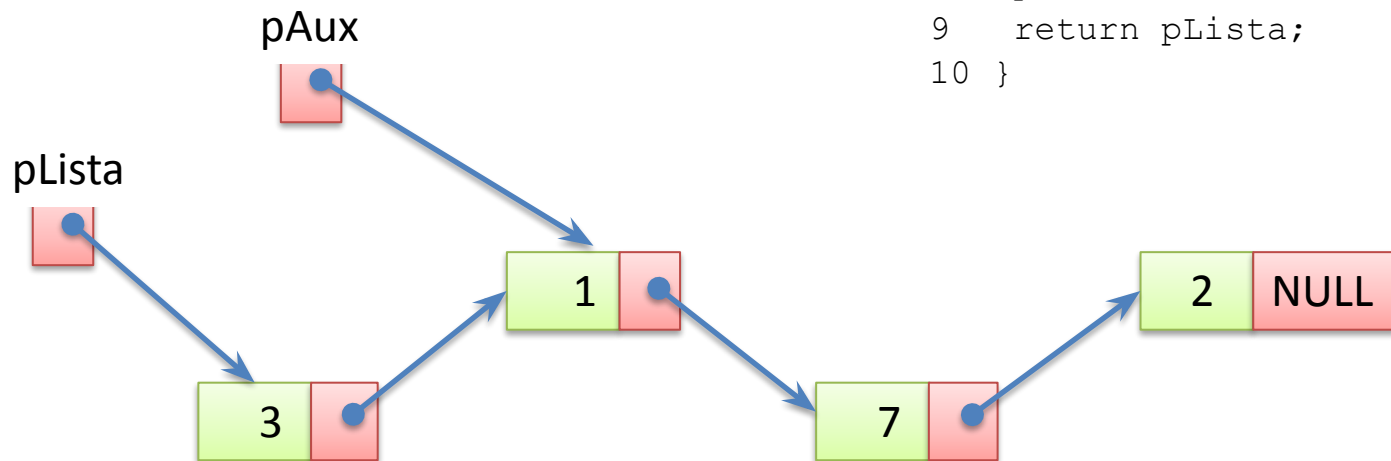
```
1 TNo *ExcluiCalda(TNo *pLista)
2 {
3     TNo *pAux;
4     pAux = pLista;
5     while (pAux->Prox->Prox != NULL)
6         pAux = pAux->Prox;
7     free(pAux->Prox);
8     pAux->Prox = NULL;
9     return pLista;
10 }
```





Lista Encadeada – Exclusão na Calda

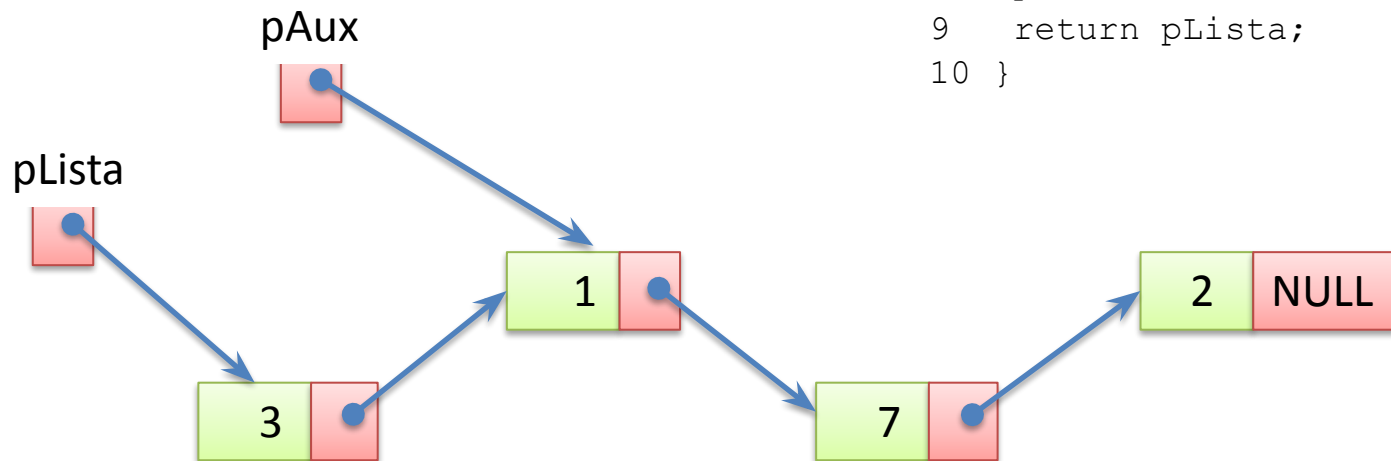
```
1 TNo *ExcluiCalda(TNo *pLista)
2 {
3     TNo *pAux;
4     pAux = pLista;
5     while (pAux->Prox->Prox != NULL)
6         pAux = pAux->Prox;
7     free(pAux->Prox);
8     pAux->Prox = NULL;
9     return pLista;
10 }
```





Lista Encadeada – Exclusão na Calda

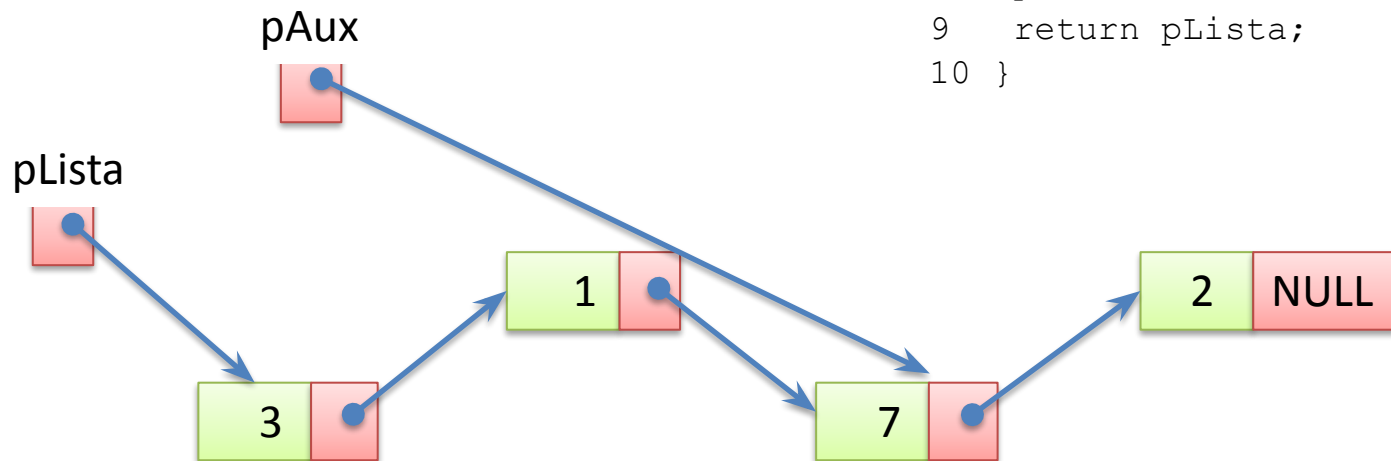
```
1 TNo *ExcluiCalda(TNo *pLista)
2 {
3     TNo *pAux;
4     pAux = pLista;
5     while (pAux->Prox->Prox != NULL)
6         pAux = pAux->Prox;
7     free(pAux->Prox);
8     pAux->Prox = NULL;
9     return pLista;
10 }
```





Lista Encadeada – Exclusão na Calda

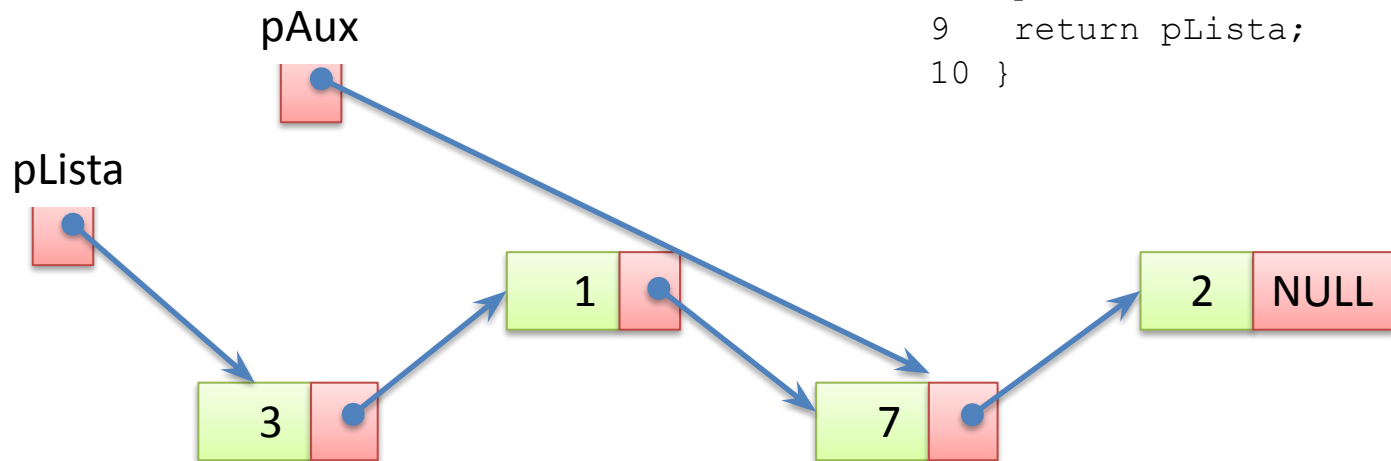
```
1 TNo *ExcluiCalda(TNo *pLista)
2 {
3     TNo *pAux;
4     pAux = pLista;
5     while (pAux->Prox->Prox != NULL)
6         pAux = pAux->Prox;
7     free(pAux->Prox);
8     pAux->Prox = NULL;
9     return pLista;
10 }
```





Lista Encadeada – Exclusão na Calda

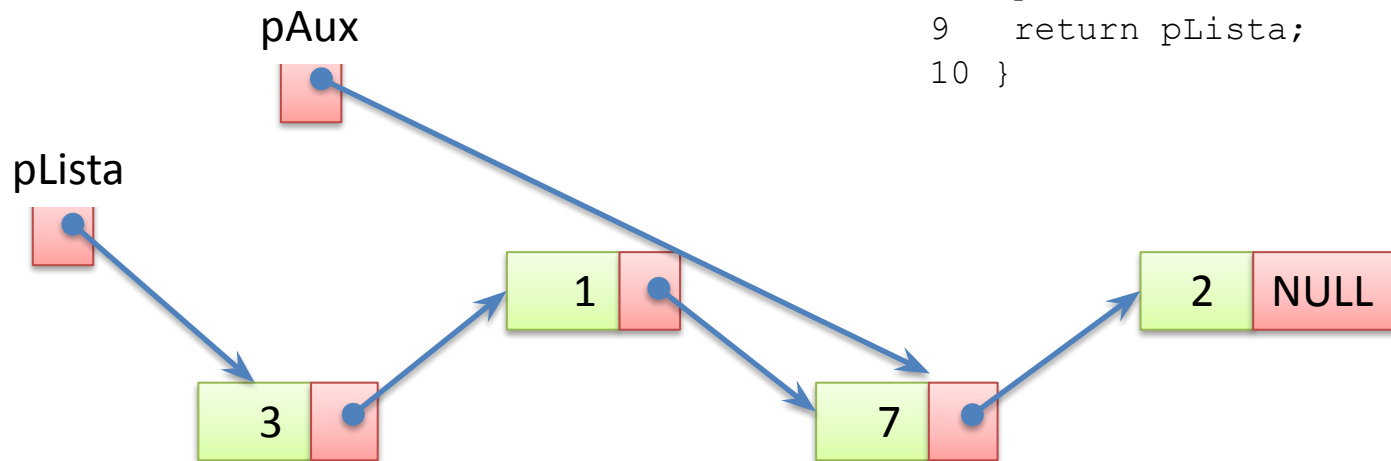
```
1 TNo *ExcluiCalda(TNo *pLista)
2 {
3     TNo *pAux;
4     pAux = pLista;
5     while (pAux->Prox->Prox != NULL)
6         pAux = pAux->Prox;
7     free(pAux->Prox);
8     pAux->Prox = NULL;
9     return pLista;
10 }
```





Lista Encadeada – Exclusão na Calda

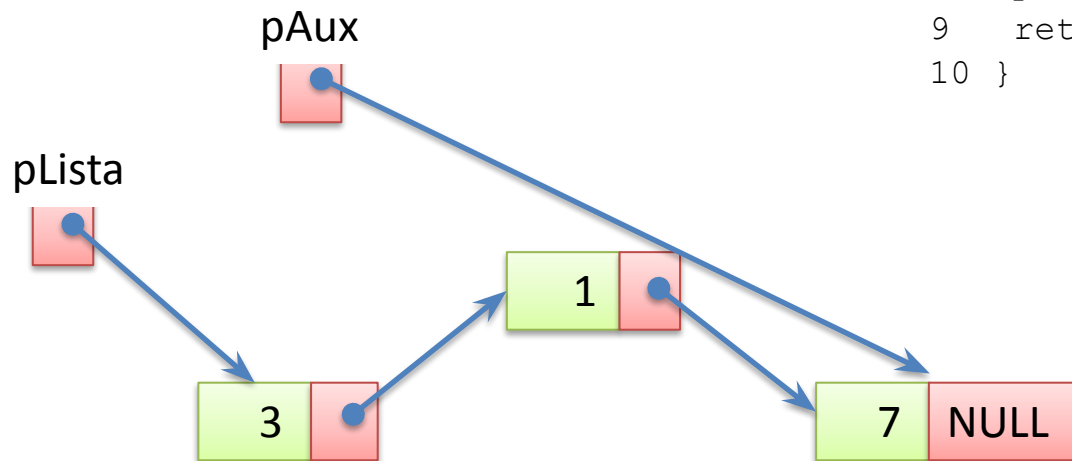
```
1 TNo *ExcluiCalda(TNo *pLista)
2 {
3     TNo *pAux;
4     pAux = pLista;
5     while (pAux->Prox->Prox != NULL)
6         pAux = pAux->Prox;
7     free (pAux->Prox) ;
8     pAux->Prox = NULL;
9     return pLista;
10 }
```





Lista Encadeada – Exclusão na Calda

```
1 TNo *ExcluiCalda(TNo *pLista)
2 {
3     TNo *pAux;
4     pAux = pLista;
5     while (pAux->Prox->Prox != NULL)
6         pAux = pAux->Prox;
7     free(pAux->Prox);
8     pAux->Prox = NULL;
9     return pLista;
10 }
```

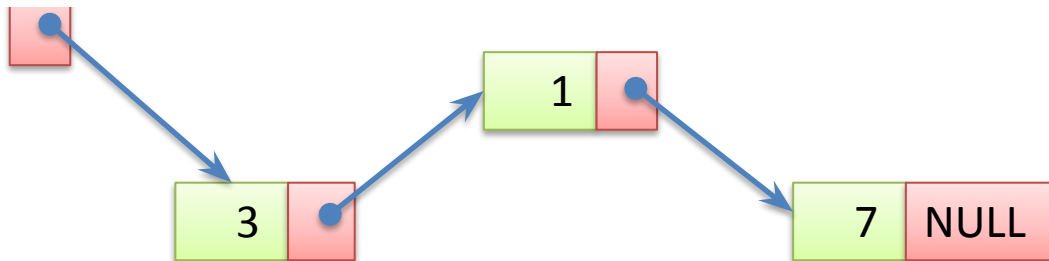




Lista Encadeada – Exclusão na Calda

```
1 TNo *ExcluiCalda(TNo *pLista)
2 {
3     TNo *pAux;
4     pAux = pLista;
5     while (pAux->Prox->Prox != NULL)
6         pAux = pAux->Prox;
7     free(pAux->Prox);
8     pAux->Prox = NULL;
9     return pLista;
10 }
```

pLista





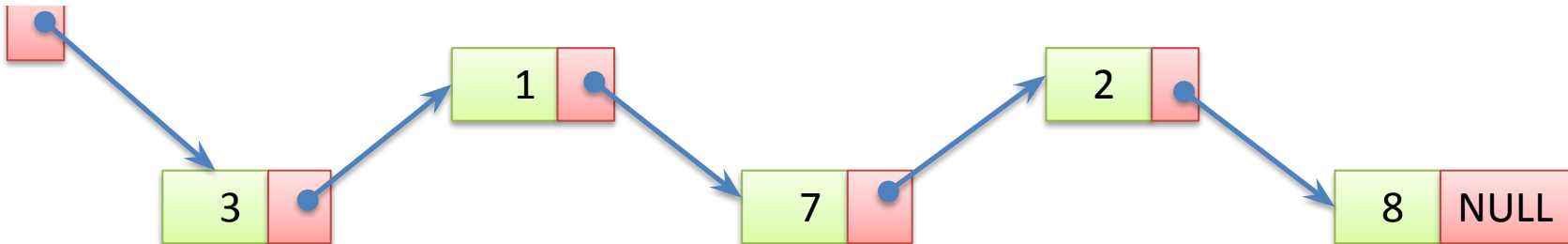
UnB - FCTE

pChave = 7

Lista Encadeada – Exclusão de Uma Chave

```
1 TNo *ExcluiChave(TNo *pLista, int pChave)
2 {
3     TNo *pAnt, *pPost;
4     pAnt = pLista;
5     while (pAnt->Prox->Numero != pChave)
6         pAnt = pAnt->Prox;
7     pPost = pAnt->Prox->Prox;
8     free(pAnt->Prox);
9     pAnt->Prox = pPost;
10    return pLista;
11 }
```

pLista

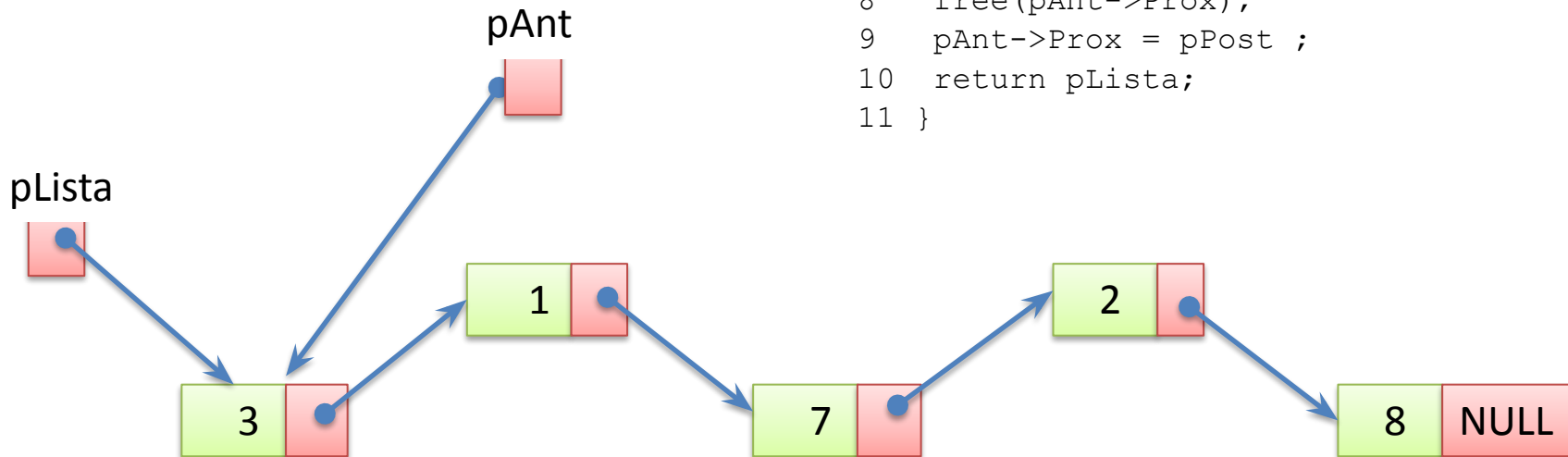




pChave = 7

Lista Encadeada – Exclusão de Uma Chave

```
1 TNo *ExcluiChave(TNo *pLista, int pChave)
2 {
3     TNo *pAnt, *pPost;
4     pAnt = pLista;
5     while (pAnt->Prox->Numero != pChave)
6         pAnt = pAnt->Prox;
7     pPost = pAnt->Prox->Prox;
8     free(pAnt->Prox);
9     pAnt->Prox = pPost ;
10    return pLista;
11 }
```

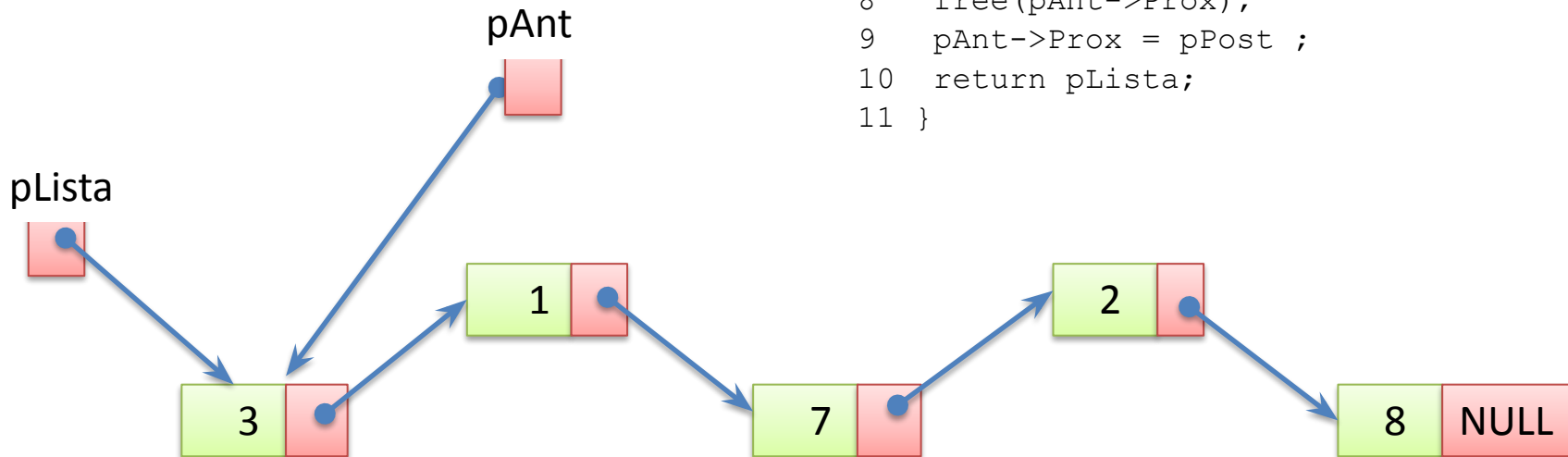




pChave = 7

Lista Encadeada – Exclusão de Uma Chave

```
1 TNo *ExcluiChave(TNo *pLista, int pChave)
2 {
3     TNo *pAnt, *pPost;
4     pAnt = pLista;
5     while (pAnt->Prox->Numero != pChave)
6         pAnt = pAnt->Prox;
7     pPost = pAnt->Prox->Prox;
8     free(pAnt->Prox);
9     pAnt->Prox = pPost ;
10    return pLista;
11 }
```



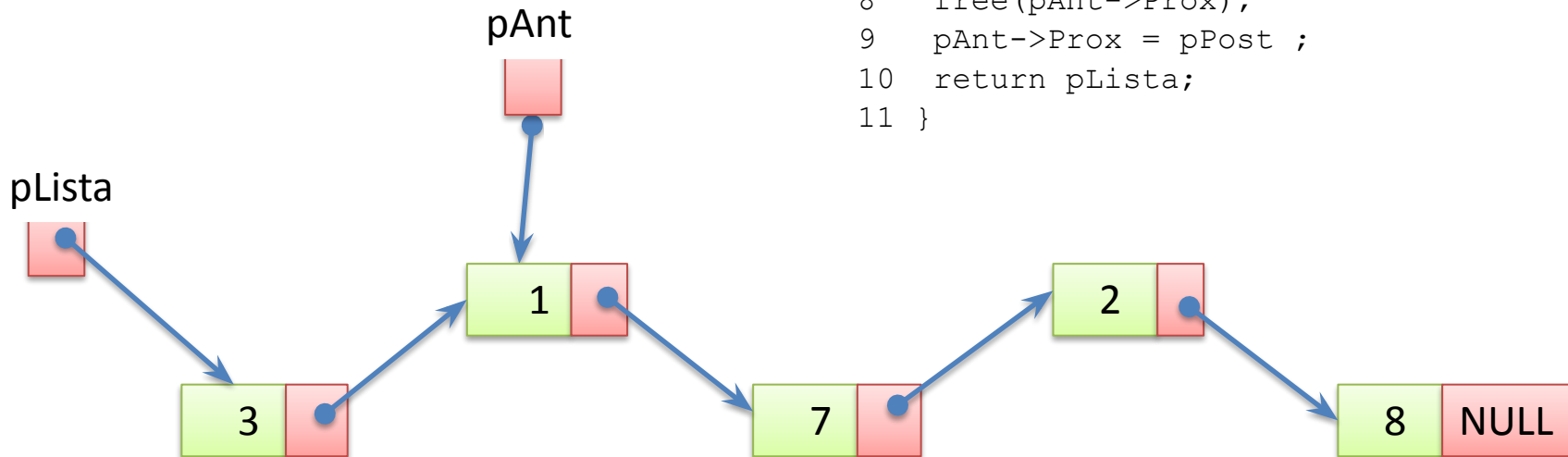


UnB - FCTE

pChave = 7

Lista Encadeada – Exclusão de Uma Chave

```
1 TNo *ExcluiChave(TNo *pLista, int pChave)
2 {
3     TNo *pAnt, *pPost;
4     pAnt = pLista;
5     while (pAnt->Prox->Numero != pChave)
6         pAnt = pAnt->Prox;
7     pPost = pAnt->Prox->Prox;
8     free(pAnt->Prox);
9     pAnt->Prox = pPost ;
10    return pLista;
11 }
```



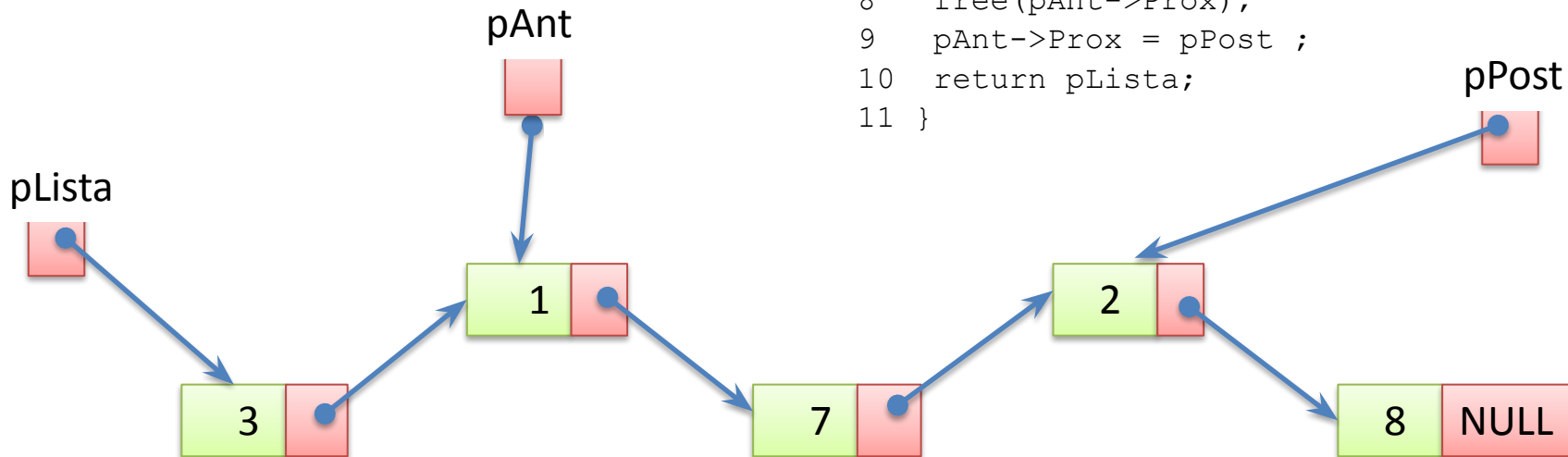


UnB - FCTE

pChave = 7

Lista Encadeada – Exclusão de Uma Chave

```
1 TNo *ExcluiChave(TNo *pLista, int pChave)
2 {
3     TNo *pAnt, *pPost;
4     pAnt = pLista;
5     while (pAnt->Prox->Numero != pChave)
6         pAnt = pAnt->Prox;
7     pPost = pAnt->Prox->Prox;
8     free(pAnt->Prox);
9     pAnt->Prox = pPost;
10    return pLista;
11 }
```



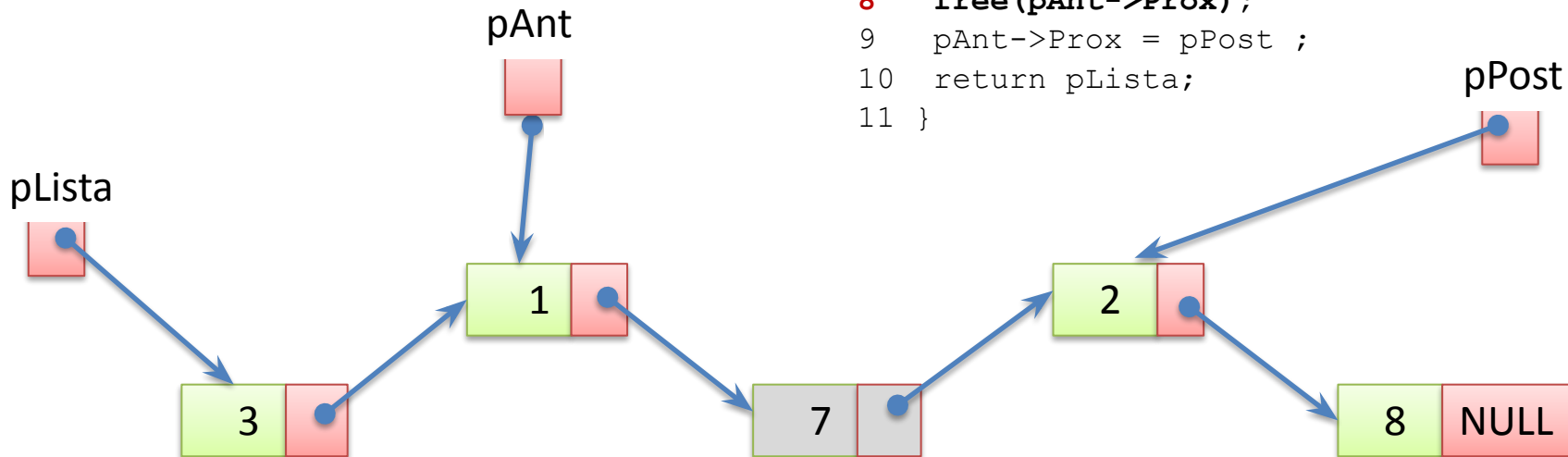


UnB - FCTE

pChave = 7

Lista Encadeada – Exclusão de Uma Chave

```
1 TNo *ExcluiChave(TNo *pLista, int pChave)
2 {
3     TNo *pAnt, *pPost;
4     pAnt = pLista;
5     while (pAnt->Prox->Numero != pChave)
6         pAnt = pAnt->Prox;
7     pPost = pAnt->Prox->Prox;
8     free (pAnt->Prox) ;
9     pAnt->Prox = pPost ;
10    return pLista;
11 }
```



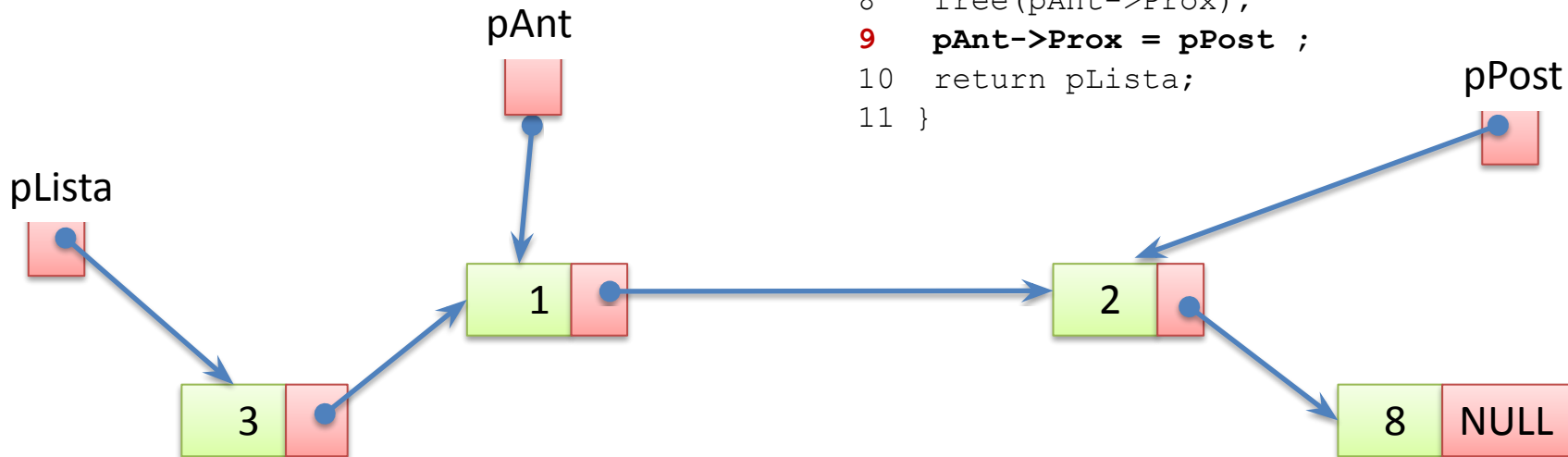


UnB - FCTE

pChave = 7

Lista Encadeada – Exclusão de Uma Chave

```
1 TNo *ExcluiChave(TNo *pLista, int pChave)
2 {
3     TNo *pAnt, *pPost;
4     pAnt = pLista;
5     while (pAnt->Prox->Numero != pChave)
6         pAnt = pAnt->Prox;
7     pPost = pAnt->Prox->Prox;
8     free(pAnt->Prox);
9     pAnt->Prox = pPost;
10    return pLista;
11 }
```





pChave = 7

Lista Encadeada – Exclusão de Uma Chave

```
1 TNo *ExcluiChave(TNo *pLista, int pChave)
2 {
3     TNo *pAnt, *pPost;
4     pAnt = pLista;
5     while (pAnt->Prox->Numero != pChave)
6         pAnt = pAnt->Prox;
7     pPost = pAnt->Prox->Prox;
8     free(pAnt->Prox);
9     pAnt->Prox = pPost ;
10    return pLista;
11 }
```

pLista





Lista Encadeada

```
struct TNo
```

```
{
```

```
    int Numero;
```

```
    Tno *Prox;
```

```
};
```

```
struct TLista
```

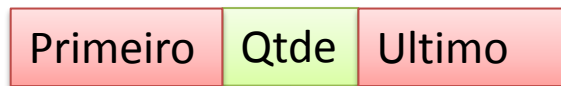
```
{
```

```
    TNo *Primeiro;
```

```
    int Qtde;
```

```
    TNo *Ultimo;
```

```
};
```





Lista Encadeada

